

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO PROJECT II

ĐỀ TÀI: XÂY DỰNG HỆ THỐNG CẢNH BÁO CHÁY THÔNG MINH

Sinh viên thực hiện: Nguyễn Đình Kiên

Mã số sinh viên: 20235356

Giảng viên hướng dẫn: PGS.TS. Trương Thị Diệu Linh

Hà Nội, tháng 06 năm 2026

MỤC LỤC

LỜI NÓI ĐẦU	6
CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI	7
1.1. Lý do chọn đề tài	7
1.2. Mục tiêu nghiên cứu	7
1.3. Công nghệ sử dụng	8
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	9
2.1. Tổng quan về mô hình YOLO	9
2.1.1. Mạng nơ-ron tích chập	9
2.1.2. Bài toán Nhận diện vật thể	10
2.1.3. Cơ chế của mô hình YOLO	10
2.1.4. Mô hình YOLOv8n và áp dụng vào hệ thống	12
2.2. Giao thức truyền tin MQTT	13
2.3. Giao thức WebSocket	14
2.4. Vi điều khiển ESP32 và ESP32-CAM	15
2.5. Các công nghệ xây dựng hệ thống máy chủ và ứng dụng	17
2.5.1. Backend: NestJS	17
2.5.2. Frontend: React & Vite	17
2.5.3. Hệ quản trị cơ sở dữ liệu PostgreSQL	17
CHƯƠNG 3: PHÂN TÍCH YÊU CẦU BÀI TOÁN	18
3.1. Mô hình hóa chức năng	18
3.1.1. Các tác nhân trong hệ thống	18
3.1.2. Các yêu cầu chức năng	19
3.1.3. Các yêu cầu phi chức năng	20
3.1.4. Biểu đồ use case tổng quan	21
3.1.5. Đặc tả use case	21
3.2. Mô hình hóa cấu trúc	29
3.3. Mô hình hóa hành vi	31
3.3.1. Sơ đồ máy trạng thái cho đối tượng camera	31
3.3.2. Sơ đồ tuần tự cho các use case	31
CHƯƠNG 4: THIẾT KẾ HỆ THỐNG	40
4.1. Thiết kế kiến trúc hệ thống	40
4.2. Thiết kế giao tiếp	41
4.2.1. Đặc tả các RESTful API	41

4.2.2.	Đặc tả giao tiếp WebSocket	42
4.2.3.	Đặc tả giao tiếp MQTT	44
4.3.	Thiết kế thiết bị vật lý (Camera)	46
4.3.1.	Sơ đồ kết nối chân phần cứng	46
4.3.2.	Quy trình hoạt động	47
4.4.	Thiết kế chương trình xử lý AI	48
4.5.	Thiết kế cơ sở dữ liệu	49
4.6.	Thiết kế giao diện	53
4.6.1.	Giao diện đăng nhập & đăng ký tài khoản	53
4.6.2.	Giao diện giám sát tổng quan	54
4.6.3.	Giao diện giám sát từng camera	55
4.6.4.	Giao diện lịch sử cảnh báo	56
4.6.5.	Giao diện quản lý phòng	57
4.6.6.	Giao diện quản lý camera	58
4.6.7.	Giao diện thông tin cá nhân	59
4.6.8.	Giao diện dành cho điện thoại	60
CHƯƠNG 5: KẾT QUẢ THỰC HIỆN		61
5.1.	Kết quả huấn luyện mô hình YOLO	61
5.2.	Thử nghiệm hệ thống	64
5.2.1.	Thử nghiệm với đám cháy giả lập	64
5.2.2.	Thử nghiệm với đám cháy thật quy mô nhỏ	67
CHƯƠNG 6: ĐÁNH GIÁ VÀ KẾT LUẬN		69
6.1.	Kết quả đã đạt được	69
6.2.	Đánh giá hệ thống	69
6.3.	Hướng phát triển và mở rộng của đề tài	70

DANH MỤC HÌNH ẢNH

2.1	Quy trình xử lý của mô hình YOLO	10
2.2	Cấu trúc mạng nơ-ron của mô hình YOLO	12
2.3	Luồng hoạt động của giao thức WebSocket	15
2.4	Sơ đồ chân ESP32	15
2.5	Sơ đồ chân ESP32-CAM	16
3.1	Biểu đồ use case tổng quan	21
3.2	Sơ đồ lớp của hệ thống	30
3.3	Sơ đồ máy trạng thái cho đối tượng Camera	31
3.4	Sơ đồ tuần tự cho Use Case 1	32
3.5	Sơ đồ tuần tự cho Use Case 2	33
3.6	Sơ đồ tuần tự cho Use Case 3	34
3.7	Sơ đồ tuần tự cho Use Case 4	35
3.8	Sơ đồ tuần tự cho Use Case 5	36
3.9	Sơ đồ tuần tự cho Use Case 6	36
3.10	Sơ đồ tuần tự cho Use Case 7	37
3.11	Sơ đồ tuần tự cho Use Case 8	38
3.12	Sơ đồ tuần tự cho Use Case 9	39
4.1	Sơ đồ kiến trúc hệ thống	40
4.2	Giao diện đăng nhập	53
4.3	Giao diện đăng ký tài khoản	54
4.4	Giao diện giám sát tổng quan	55
4.5	Giao diện giám sát từng camera	56
4.6	Giao diện lịch sử cảnh báo	57
4.7	Giao diện quản lý phòng	57
4.8	Giao diện quản lý camera	58
4.9	Giao diện chỉnh sửa camera	59
4.10	Giao diện thông tin cá nhân	59
4.11	Giao diện dành cho điện thoại	60
5.1	Dữ liệu huấn luyện trên Roboflow	61
5.2	Kết quả huấn luyện mô hình YOLO	62
5.3	Đường cong PR Curve	63
5.4	Ma trận nhầm lẫn của mô hình	64

5.5	Hệ thống phát cảnh báo	65
5.6	Cảnh báo đầy trên điện thoại	65
5.7	Camera vật lý cũng phát tín hiệu cảnh báo	66
5.8	Hình ảnh đám cháy ghi nhận trên hệ thống	66
5.9	Giao diện người dùng phát cảnh báo	67
5.10	Camera phát cảnh báo đèn đỏ	68

DANH SÁCH BẢNG

3.1	Đặc tả Use Case 01	22
3.2	Đặc tả Use Case 02	23
3.3	Đặc tả Use Case 03	24
3.4	Đặc tả Use Case 04	26
3.5	Đặc tả Use Case 05	27
3.6	Đặc tả Use Case 06	27
3.7	Đặc tả Use Case 07	28
3.8	Đặc tả Use Case 08	29
3.9	Đặc tả Use Case 09	29
4.1	Danh sách các Endpoint RESTful API	42
4.2	Danh mục đặc tả các sự kiện truyền phát qua WebSocket	44
4.3	Danh mục đặc tả các Topic MQTT hệ thống	46
4.4	Bảng sơ đồ kết nối chân phần cứng	47
4.5	Đặc tả chi tiết bảng users	50
4.6	Đặc tả chi tiết bảng rooms	50
4.7	Đặc tả chi tiết bảng cameras	52
4.8	Đặc tả chi tiết bảng fire_alerts	53

LỜI NÓI ĐẦU

Trong những năm gần đây, tình hình cháy nổ tại các khu công nghiệp, khu dân cư, đặc biệt là các khu nhà tập thể, chung cư, nhà trọ đang có những diễn biến hết sức phức tạp, gây ra nhiều thiệt hại to lớn về người và tài sản. Việc phát hiện và cảnh báo cháy kịp thời ngay từ những giây phút đầu tiên là yếu tố mang tính quyết định để giảm thiểu tối đa các hậu quả đáng tiếc.

Ngày nay, trí tuệ nhân tạo (AI) - đặc biệt là lĩnh vực thị giác máy tính (Computer Vision) và internet vạn vật (IoT) - đã mở ra những hướng đi mới trong việc giải quyết các bài toán an toàn an ninh. Xuất phát từ yêu cầu thực tiễn đó, ý tưởng về xây dựng một hệ thống cảnh báo cháy áp dụng trí tuệ nhân tạo kết hợp với internet vạn vật được ra đời nhằm giải quyết bài toán này.

Mục tiêu chính của đề tài là nghiên cứu và phát triển một hệ thống có khả năng thu nhận luồng video theo thời gian thực từ các camera giám sát, sử dụng các mô hình học sâu (Deep Learning), cũng như phối hợp với các cảm biến vật lý truyền thống để tự động nhận diện các dấu hiệu của khói và lửa. Khi phát hiện sự cố, hệ thống AI sẽ lập tức giao tiếp với máy chủ trung tâm để kích hoạt quy trình báo động, giúp người quản lý có thể đưa ra phương án xử lý nhanh chóng và chính xác nhất.

Để hoàn thành được dự án này, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến cô Trương Thị Diệu Linh, người đã luôn tận tình định hướng, chỉ bảo và cung cấp những kiến thức quý báu cho em trong suốt quá trình nghiên cứu và thực hiện đề tài.

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Lý do chọn đề tài

Hiện nay, công tác phòng cháy chữa cháy chủ yếu vẫn dựa vào các hệ thống báo cháy truyền thống sử dụng cảm biến khói hoặc nhiệt độ. Dù phổ biến, các hệ thống này tồn tại một số nhược điểm như: thời gian phản hồi chậm (phải đợi khói hoặc nhiệt lan tới cảm biến), diện tích giám sát hẹp, gây ảnh hưởng tới quá trình giám sát và phát hiện sớm đám cháy.

Để khắc phục những hạn chế đó, một giải pháp đang được phổ biến trong thời gian gần đây, đó là áp dụng trí tuệ nhân tạo, cụ thể là công nghệ xử lý ảnh. Công nghệ nhận diện hình ảnh có thể phát hiện ngọn lửa hoặc khói ngay từ khi mới bùng phát thông qua luồng video trực tiếp từ camera giám sát. Hơn nữa, việc kết hợp chéo dữ liệu từ AI và các cảm biến môi trường (nhiệt độ, nồng độ khí gas,...) sẽ giúp hệ thống đưa ra quyết định cảnh báo chính xác hơn, loại bỏ tình trạng báo động giả. Từ những yêu cầu thực tiễn đó, đề tài "Xây dựng hệ thống cảnh báo cháy thông minh" đã được nghiên cứu và phát triển.

1.2 Mục tiêu nghiên cứu

Mục tiêu cốt lõi của đề tài là xây dựng một hệ thống cảnh báo cháy toàn diện, hoạt động theo thời gian thực với khả năng nhận diện độ chính xác cao và quản lý tập trung. Cụ thể, đề tài sẽ nghiên cứu và triển khai mô hình phát hiện vật thể (YOLO) để nhận diện dấu hiệu cháy qua luồng video streaming từ camera giám sát. Đồng thời, xây dựng luồng giao tiếp dữ liệu mượt mà, độ trễ thấp thông qua giao thức MQTT để thu thập dữ liệu từ các node cảm biến vật lý và thiết bị camera; cũng như xây dựng máy chủ tập trung để quản lý toàn bộ hệ thống, cũng như xử lý logic quan trọng hệ thống, bao gồm cả xử lý hình ảnh AI và phát cảnh báo tới người dùng. Cuối cùng, xây dựng một giao diện người dùng cơ bản, thân thiện, tích hợp cảnh báo đầy đủ để thông báo lập tức cho phía người dùng.

1.3 Công nghệ sử dụng

Hệ thống sử dụng các phương pháp và công cụ hiện đại nhất. Cụ thể, khối trí tuệ nhân tạo (AI Worker) được lập trình bằng Python, sử dụng thư viện Ultralytics (YOLO) để huấn luyện và chạy mô hình nhận diện lửa, sử dụng OpenCV để xử lý và trích xuất khung hình từ luồng luồng video. Khối này cũng đảm nhiệm nhiệm vụ lưu lại hình ảnh/video bằng chứng khi phát hiện có đám cháy, để gửi lại cho người dùng.

Khối máy chủ trung tâm, được xây dựng dựa trên framework NestJS (Node.js). Cơ sở dữ liệu sử dụng PostgreSQL, kết nối với ứng dụng backend thông qua TypeORM để lưu trữ thông tin thiết bị, lịch sử cảnh báo và thông tin hình ảnh/video sự cố. Để các phần cứng có thể giao tiếp với nhau, hệ thống sử dụng giao thức MQTT

Ứng dụng người dùng được xây dựng bằng React và Vite, tích hợp Firebase Cloud Messaging (FCM) để gửi Push Notification cảnh báo khẩn cấp tới thiết bị di động.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về mô hình YOLO

YOLO (You Only Look Once) là một mô hình mạng nơ-ron tích chập, được xây dựng để phục vụ cho bài toán nhận diện vật thể. Được giới thiệu lần đầu bởi nhóm tác giả đến từ đại học Washington vào năm 2016, cho đến nay đây là một trong những mô hình nổi tiếng nhất ở trong lĩnh vực Thị giác máy tính nói riêng và trong Trí tuệ nhân tạo nói chung.

2.1.1 Mạng nơ-ron tích chập

Mạng nơ-ron tích chập (CNN - Convolutional Neural Network) là một trong những kiến trúc học sâu (Deep Learning) tiên tiến và hiệu quả nhất hiện nay đối với các bài toán thuộc lĩnh vực thị giác máy tính. Khác với các mạng nơ-ron truyền thống thường bị quá tải dữ liệu và mất đi thông tin không gian khi phẳng hóa ảnh đầu vào, mạng nơ-ron tích chập được thiết kế để tự động trích xuất các đặc trưng mang tính cấu trúc hình học của bức ảnh một cách trực tiếp từ các ma trận điểm ảnh.

Mạng nơ-ron tích chập sử dụng các lớp đặc trưng sau để tự động trích xuất đặc trưng của ảnh:

- **Lớp tích chập (Convolutional Layer):** Sử dụng các bộ lọc (Kernels/Filters) trượt qua toàn bộ bề mặt ảnh để phát hiện các đặc trưng từ cơ bản (cạnh, góc, màu sắc) đến phức tạp (hình dáng khối, ngọn lửa).
- **Lớp pooling (Pooling Layer):** Thực hiện giảm chiều không gian của ma trận đặc trưng (thường dùng Max Pooling) để giữ lại các thông tin nổi bật nhất, đồng thời giảm khối lượng tính toán cho hệ thống.
- **Lớp kết nối đầy đủ (Fully Connected Layer):** Nằm ở cuối mạng mạng, có nhiệm vụ phẳng hóa dữ liệu và đưa ra kết quả phân loại (Classification) hoặc dự đoán cuối cùng.

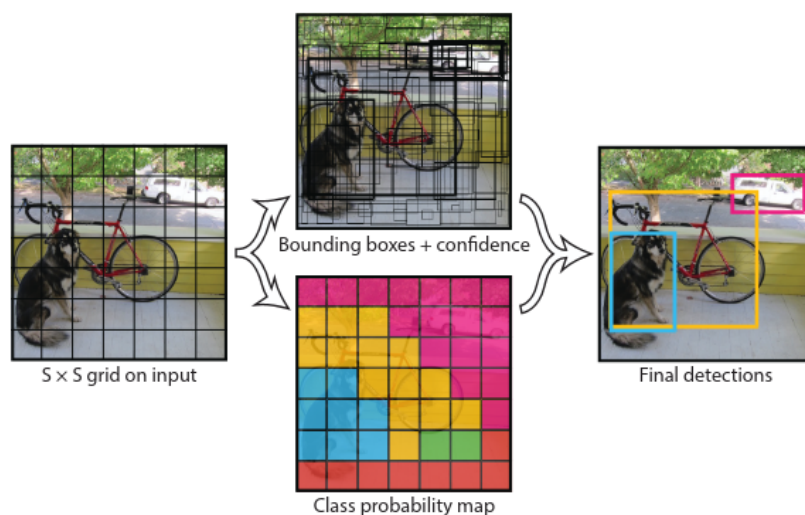
2.1.2 Bài toán Nhận diện vật thể

Nhận diện vật thể là một trong những bài toán cốt lõi của lĩnh vực thị giác máy tính. Bài toán này đòi hỏi phải giải quyết đồng thời hai nhiệm vụ: phân loại vật thể trong ảnh thuộc nhãn nào (ví dụ: lửa, khói...); và xác định chính xác vị trí của vật thể đó trong bức ảnh.

Trước khi YOLO ra đời, các mô hình phát hiện vật thể thường sử dụng phương pháp hai giai đoạn như R-CNN hay Faster R-CNN. Các mô hình này đầu tiên sẽ đề xuất các vùng có khả năng chứa vật thể, sau đó mới tiến hành phân loại trên các vùng đó. Nhược điểm của chúng là tốc độ tính toán chậm, khó đáp ứng được yêu cầu thời gian thực mà nhiều bài toán nhận diện vật thể hiện nay đòi hỏi, trong đó có bài toán cảnh báo cháy mà chúng ta đang nghiên cứu.

Trong khi đó, mô hình YOLO áp dụng kiến trúc một giai đoạn, đúng như tên gọi của nó: "You Only Look Once". Mô hình coi bài toán nhận diện vật thể là một bài toán hồi quy duy nhất, đi thẳng từ điểm ảnh đến tọa độ hộp giới hạn và xác suất phân loại chỉ trong một lần lan truyền qua mạng nơ-ron. Điều này giúp YOLO đạt được tốc độ vượt trội so với các mô hình trước đây.

2.1.3 Cơ chế của mô hình YOLO



Hình 2.1: Quy trình xử lý của mô hình YOLO

Chia lưới ảnh đầu vào

YOLO chia bức ảnh đầu vào thành một lưới kích thước $S \times S$ (ví dụ: 13×13 , $26 \times 26 \dots$). Nếu tâm của một ngọn lửa rơi vào một ô lưới cụ thể, ô đó sẽ chịu trách nhiệm phát hiện ngọn lửa này.

Dự đoán đồng thời qua mạng tích chập

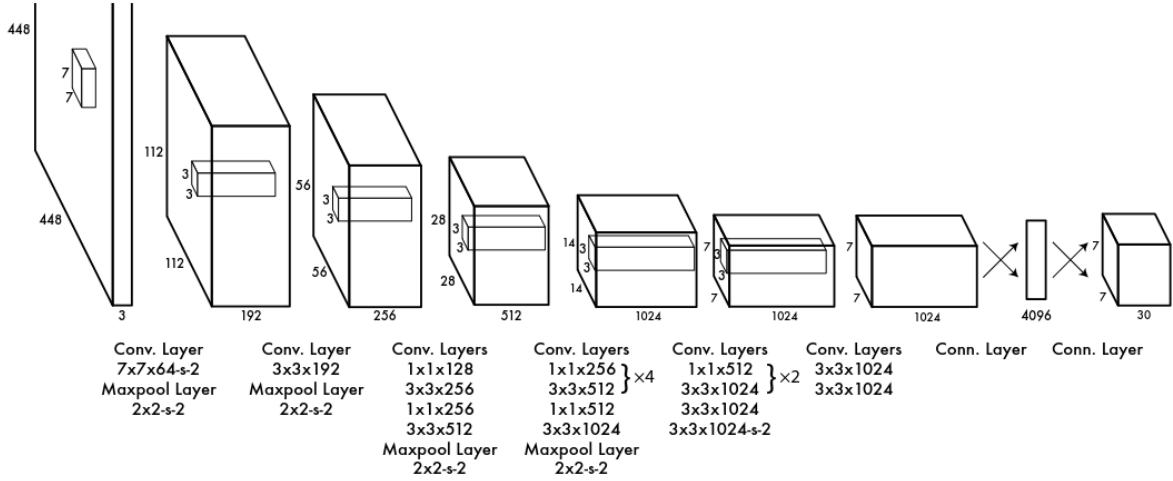
Mạng nơ-ron tích chập duy nhất của YOLO, dựa trên mạng GoogLeNet với 24 lớp chập và 2 lớp kết nối đầy đủ, sẽ xử lý toàn bộ hình ảnh trong một lượt lan truyền duy nhất. Mỗi ô lưới sẽ dự đoán ra B hộp giới hạn cùng với điểm tin cậy cho các hộp đó. Kết quả dự đoán sẽ được trả về với 5 thông số như sau:

- **Tọa độ (x, y) :** Thể hiện vị trí tâm của hộp giới hạn, mang giá trị tương đối so với ranh giới của chính ô lưới đó.
- **Kích thước (w, h) :** Thể hiện chiều rộng và chiều cao của hộp giới hạn, mang giá trị tương đối so với toàn bộ kích thước của bức ảnh.
- **Điểm tin cậy (Confidence):** Phản ánh mức độ tự tin của mô hình về việc có một đối tượng đang nằm trong hộp dự đoán (Predicted Bounding Box) đó, đồng thời đo lường độ chính xác so với hộp thực tế (Ground-truth Bounding Box). Điểm tin cậy được tính theo công thức:

$$Confidence = Pr(Object) \times IOU_{pred}^{truth} \quad (2.1)$$

Trong đó, $Pr(Object)$ là xác suất có một đối tượng tồn tại trong ô lưới đó. Nếu không có đối tượng nào trong ô, điểm độ tin cậy bằng 0. Nếu có đối tượng, mô hình kỳ vọng điểm độ tin cậy sẽ bằng đúng giá trị IOU . IOU (Intersection over Union) là hệ số đo lường độ trùng khớp, tính bằng diện tích phần giao nhau chia cho diện tích phần hợp nhất giữa hộp giới hạn do mô hình dự đoán (Predicted Bounding Box) và hộp giới hạn thực tế (Ground-truth Bounding Box).

Cùng lúc đó, mỗi ô cũng sẽ đưa ra C xác suất $Pr(Class_i|Object)$, tức là xác suất để đối tượng trong ô đó thuộc về lớp i với điều kiện là có đối tượng tồn tại trong ô đó. YOLO chỉ dự đoán đúng một tập xác suất lớp duy nhất cho mỗi ô lưới, bất kể ô lưới đó dự đoán bao nhiêu hộp B .



Hình 2.2: Cấu trúc mạng nơ-ron của mô hình YOLO

Tính toán điểm số đặc trưng cho từng hộp

Tại thời điểm kiểm tra, để biết một hộp giới hạn cụ thể có độ tự tin đối với một lớp đối tượng nào đó là bao nhiêu, mô hình thực hiện phép nhân:

$$Pr(Class_i|Object) \times Pr(Object) \times IoU_{pred}^{truth} = Pr(Class_i) \times IoU_{pred}^{truth} \quad (2.2)$$

Kết quả trả về sẽ là điểm tin cậy cho từng lớp (class-specific confidence score). Điểm số này thể hiện đồng thời cả hai điều: xác suất lớp đó xuất hiện trong hộp và mức độ hộp dự đoán khớp với vật thể tốt như thế nào.

Giai đoạn hậu xử lý

Do một đối tượng lớn hoặc một đối tượng nằm gần ranh giới giữa các ô lưới có thể bị dự đoán bởi nhiều ô lưới cùng lúc, YOLO sử dụng thuật toán triệt tiêu phi cực đại (Non-Maximum Suppression - NMS) nhằm loại bỏ các dự đoán trùng lặp để giữ lại hộp giới hạn chính xác nhất, giúp tăng thêm 2-3% độ chính xác trung bình (mAP).

2.1.4 Mô hình YOLOv8n và áp dụng vào hệ thống

YOLOv8n là phiên bản nhẹ và nhanh nhất trong họ mô hình YOLOv8 được phát triển bởi Ultralytics vào năm 2023. Mô hình này được thiết kế tối ưu hóa đặc biệt cho các thiết bị phần cứng có cấu hình giới hạn hoặc các hệ thống cần tốc độ xử lý nhanh (như Edge AI, thiết bị IoT, hay tiến trình chạy Real-time trên CPU/GPU phổ thông).

Mặc dù vậy, YOLOv8n vẫn được trang bị những nâng cấp kiến trúc tiên tiến nhất của thế hệ YOLOv8, so với các thế hệ cũ trước đây:

- **Không sử dụng hộp neo:** YOLOv8n áp dụng cơ chế xử lý không dùng hộp neo (anchor-free) thay thế cho hệ ô lưới như phân tích ở trên đối với các thế hệ cũ, giúp giảm thiểu độ phức tạp tính toán, loại bỏ các siêu tham số thừa và tăng khả năng thích ứng với các vật thể có tỷ lệ kích thước đa dạng.
- **Mạng CSPNet kết hợp với FPN+PAN:** Giúp trích xuất đặc trưng hình ảnh hiệu quả, giảm sự dư thừa tính toán nhưng vẫn giữ nguyên được sức mạnh phân tích, đồng thời giúp phát hiện tốt hơn các vật thể ở nhiều tỷ lệ và kích cỡ khác nhau.
- **Huấn luyện với độ chính xác hỗn hợp:** Giúp tăng tốc thời gian huấn luyện và giảm tiêu thụ bộ nhớ mà vẫn duy trì được chất lượng mô hình.

Có thể thấy, việc lựa chọn YOLOv8n thay vì các mô hình cũ hoặc lớn hơn là một chiến lược tối ưu cho hệ thống cảnh báo cháy này. Nó giải quyết hoàn hảo bài toán cân bằng giữa tốc độ xử lý và độ chính xác, giúp hệ thống vừa đảm bảo phát hiện cháy ngay lập tức ở giây đầu tiên, vừa có thể triển khai dễ dàng trên các hệ thống máy chủ đám mây giá rẻ hoặc home server tại nhà mà vẫn chạy mượt mà theo thời gian thực.

2.2 Giao thức truyền tin MQTT

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền thông điệp hoạt động ở tầng ứng dụng của mô hình mạng TCP/IP. Được thiết kế bởi Andy Stanford - Clark (IBM) và Arlen Nipper (EUROTECH) vào năm 1999, MQTT nhắm tới mục đích kết nối các thiết bị phần cứng từ xa có cấu hình tài nguyên giới hạn (băng thông thấp, dung lượng bộ nhớ nhỏ, năng lượng pin ít) trong hệ sinh thái Internet vạn vật (IoT).

Khác với giao thức HTTP truyền thông sử dụng mô hình Client/Server, MQTT hoạt động dựa trên cơ chế Publish/Subscribe (Xuất bản/Đăng ký) thông qua một thiết bị môi giới trung tâm gọi là MQTT Broker. Cơ chế này giúp ngắt kết nối hoàn toàn về mặt không gian và thời gian giữa thiết bị gửi dữ liệu và thiết bị nhận dữ liệu.

Các thành phần cốt lõi trong kiến trúc này bao gồm:

- **MQTT Broker:** Là máy chủ trung tâm, chịu trách nhiệm nhận tất cả các thông điệp từ bên gửi, lọc thông điệp theo chủ đề, và phân phối chính xác thông điệp đó đến tất cả các bên nhận đã đăng ký.
- **Publisher (Bên xuất bản):** Là thiết bị hoặc tiến trình thực hiện gửi dữ liệu lên Broker kèm theo một chủ đề cụ thể.
- **Subscriber (Bên đăng ký):** Là thiết bị hoặc tiến trình đăng ký nhận dữ liệu từ một hoặc nhiều chủ đề trên Broker.
- **Topic (Chủ đề):** Là một chuỗi ký tự được phân cấp bằng dấu gạch chéo (ví dụ: rooms/room_1/sensors hoặc cameras/cam_2/status). Topic hoạt động như một bộ lọc đường dẫn để Broker biết cần phải định tuyến thông điệp đi đâu.

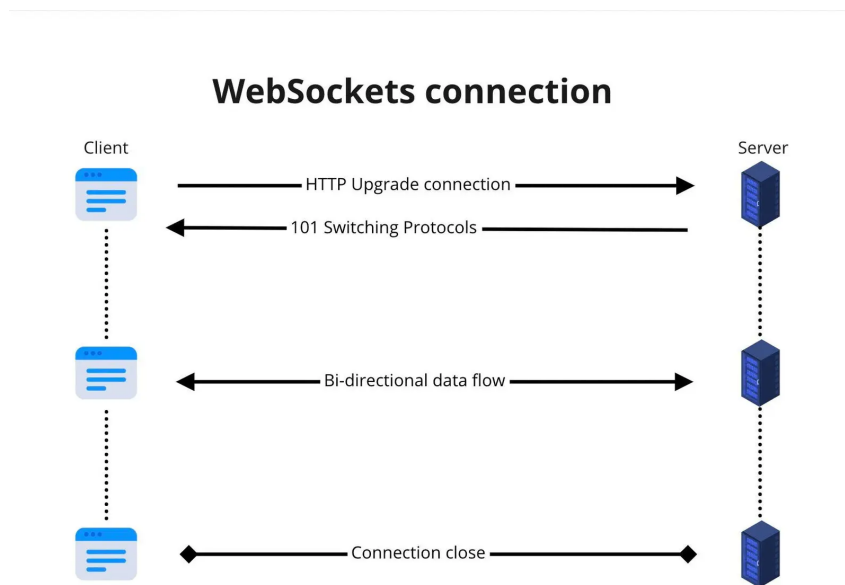
Ngoài ra, MQTT còn có cơ chế Nhịp tim (Keep Alive) & Di chúc (Last Will and Testament), cho phép broker tự động phát hiện thiết bị đột ngột bị mất kết nối. Khi đó, broker sẽ tự động gửi đi một thông điệp "di chúc" để hệ thống Web/App lập tức hiển thị trạng thái của thiết bị đó, giúp người quản lý kịp thời bảo trì.

2.3 Giao thức WebSocket

WebSocket là giao thức mạng cho phép thiết lập kênh truyền thông hai chiều toàn song công (Full-Duplex) liên tục giữa client và server trên một kết nối TCP duy nhất. Khác với giao thức HTTP truyền thống hoạt động theo cơ chế Request-Response (Client hỏi - Server trả lời), WebSocket giữ cho đường truyền luôn mở sau khi kết nối thành công.

Quy trình hoạt động của WebSocket gồm hai giai đoạn chính:

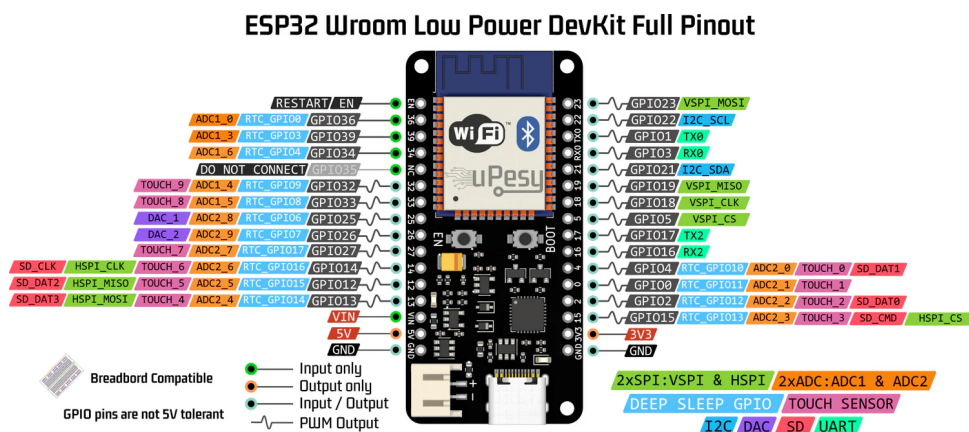
- **Bắt tay:** Client gửi một yêu cầu HTTP đặc biệt chứa tiêu đề Upgrade: websocket. Nếu Server chấp nhận, hệ thống sẽ trả về trạng thái HTTP 101 Switching Protocols, chính thức chuyển đổi kết nối HTTP thành kết nối WebSocket.
- **Truyền tải dữ liệu:** Sau khi bắt tay thành công, cả Client và Server đều có thể chủ động gửi thông điệp dạng văn bản (JSON) hoặc nhị phân sang cho nhau bất cứ lúc nào với dung lượng gói tin siêu nhẹ (do đã loại bỏ phần Header rườm rà của HTTP).



Hình 2.3: Luồng hoạt động của giao thức WebSocket

Trong hệ thống cảnh báo cháy này, WebSocket được triển khai để phục vụ cập nhật dữ liệu thời gian thực, truyền phát luồng video liên tục dưới dạng các chuỗi ảnh Proxy dạng Base64, và cảnh báo hỏa hoạn lập tức, để người dùng kịp thời ứng phó.

2.4 Vi điều khiển ESP32 và ESP32-CAM

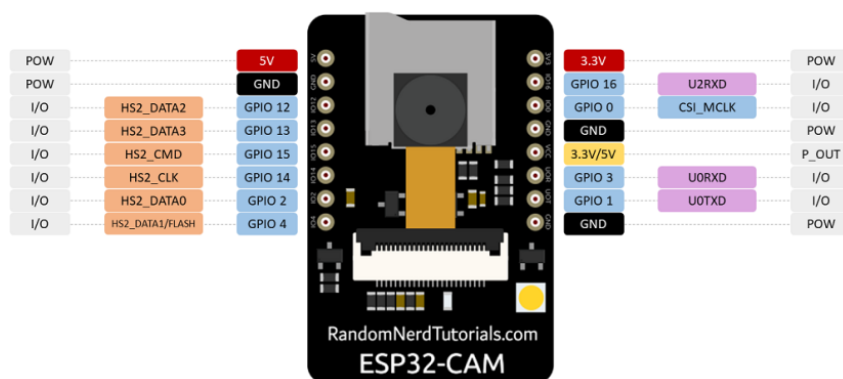


Hình 2.4: Sơ đồ chân ESP32

ESP32 là một dòng vi điều khiển hệ thống trên chip (SoC) giá rẻ, tiêu thụ năng lượng thấp, được tích hợp sẵn kết nối Wi-Fi và Bluetooth hai chế độ (Classic/BLE).

Nhờ sở hữu bộ vi xử lý lõi kép Tensilica Xtensa 32-bit LX6 hoạt động với xung nhịp lên tới 240 MHz, ESP32 cung cấp hiệu năng tính toán mạnh mẽ cho các tác vụ xử lý tại biên.

ESP32-CAM là một bo mạch phát triển kích thước siêu nhỏ dựa trên chip ESP32, được tích hợp sẵn một camera OV2640 (độ phân giải 2 Megapixel) và khe cắm thẻ nhớ MicroSD. Bo mạch này kế thừa toàn bộ sức mạnh kết nối không dây của ESP32 như trên, nhưng được tối ưu hóa riêng cho các ứng dụng xử lý và truyền tải hình ảnh giá rẻ.



Hình 2.5: Sơ đồ chân ESP32-CAM

Trong hệ thống cảnh báo cháy này, ESP32 đóng vai trò thiết bị biên, hoạt động tại hiện trường với các nhiệm vụ: thu thập dữ liệu cảm biến, đóng gói và xuất bản lên MQTT Broker gửi cho server thông qua kết nối Wi-Fi; và lắng nghe lệnh từ server để lập tức kích hoạt còi hú và đèn LED báo động khi có sự cố hỏa hoạn xảy ra. ESP32-CAM cũng thực hiện các công việc tương tự, nhưng đảm nhận thêm vai trò đó là tạo luồng video stream để gửi hình ảnh số dưới định dạng HTTP MJPEG liên tục về cho tiến trình AI, phục vụ phân tích hình ảnh.

Trong môi trường thử nghiệm của đồ án này, cụm thiết bị ESP32 (không có camera) sẽ sử dụng camera của laptop để mô phỏng cho camera của cụm thiết bị đó.

2.5 Các công nghệ xây dựng hệ thống máy chủ và ứng dụng

2.5.1 Backend: NestJS

NestJS là một framework Node.js phát triển các ứng dụng máy chủ Backend với hiệu năng cao, có khả năng mở rộng mạnh mẽ. Được xây dựng trên nền tảng TypeScript, NestJS kế thừa tư duy kiến trúc chặt chẽ của Angular, giúp lập trình viên quản lý mã nguồn thông qua cơ chế Dependency Injection (Tiêm phụ thuộc) và cấu trúc phân tầng Modular rõ ràng.

NestJS cung cấp các bộ điều khiển HTTP (Controllers) để xử lý RESTful API, tích hợp bộ chuyển đổi MQTT để giao tiếp với phần cứng/AI, và quản lý các cổng kết nối (Gateways) để truyền phát WebSocket. Đồng thời, NestJS cũng kết hợp chặt chẽ với TypeORM để tương tác trực tiếp với cơ sở dữ liệu thông qua các đối tượng Entity có cấu trúc.

2.5.2 Frontend: React & Vite

React (hay ReactJS) là một thư viện JavaScript mã nguồn mở được phát triển bởi Meta, chuyên dùng để xây dựng giao diện người dùng dạng ứng dụng một trang (SPA). React hoạt động dựa trên kiến trúc Component và cơ chế Virtual DOM giúp tối ưu hóa hiệu năng render hình ảnh khi dữ liệu thay đổi. Đi kèm với đó, Vite được lựa chọn làm công cụ đóng gói và biên dịch thế hệ mới nhờ tốc độ khởi chạy siêu nhanh thông qua cơ chế Native ESM.

Trong hệ thống này, React kết hợp Vite đảm nhận vai trò xây dựng ứng dụng người dùng ở tầng Client.

2.5.3 Hệ quản trị cơ sở dữ liệu PostgreSQL

PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở mạnh mẽ (ORDBMS). Nó nổi tiếng với độ tin cậy cao, kiến trúc an toàn, tuân thủ nghiêm ngặt các tính chất ACID (Atomicity, Consistency, Isolation, Durability) và khả năng xử lý tốt các truy vấn dữ liệu phức tạp với hiệu năng cao.

Trong hệ thống này, PostgreSQL chịu trách nhiệm lưu trữ toàn bộ dữ liệu có cấu trúc của hệ thống, ví dụ như thông tin tài khoản người dùng, cấu hình ngưỡng cảnh báo của từng thiết bị Camera, sơ đồ phân tách phòng/khu vực, lịch sử hỏa hoạn và danh sách các mã FCM Token nhận thông báo.

CHƯƠNG 3: PHÂN TÍCH YÊU CẦU BÀI TOÁN

3.1 Mô hình hóa chức năng

3.1.1 Các tác nhân trong hệ thống

Hệ thống được thiết kế để phục vụ hai tác nhân chính, đó là tác nhân con người, cụ thể ở đây là người dùng (User), và tác nhân hệ thống, cụ thể là cụm thiết bị camera tích hợp cảm biến vật lý.

Người dùng (User)

Đây là tác nhân con người tương tác qua giao diện ứng dụng web. Mỗi một user có thể đăng ký được một hoặc nhiều camera khác nhau, cũng như cấu hình thông tin và các thông số của camera. Đồng thời, người dùng có thể đăng ký một hoặc nhiều phòng, để dễ dàng quản lý khi có nhiều camera. Cuối cùng, người dùng sẽ có thể theo dõi hình ảnh camera cũng như cảm biến theo thời gian thực, và nhận được cảnh báo ngay tức thì khi camera phát hiện đám cháy xảy ra.

Camera giám sát tích hợp cảm biến IoT

Là tác nhân phần cứng ngoại vi tương tác với máy chủ thông qua giao thức MQTT, camera giám sát tích hợp cảm biến IoT (ở đây gọi chung là camera) đóng vai trò là một thiết bị giám sát an toàn toàn diện tại hiện trường. Thiết bị thực hiện đồng thời hai nhiệm vụ: liên tục cung cấp dữ liệu luồng video trực tuyến; định kỳ thu thập, gửi các thông số cảm biến vật lý về máy chủ; và nhận tín hiệu từ máy chủ để phát cảnh báo cháy bằng còi báo.

Hệ thống Firebase Cloud Messaging (FCM)

Là tác nhân hệ thống bên thứ ba liên kết với máy chủ Backend, FCM có nhiệm vụ tiếp nhận yêu cầu đẩy dữ liệu báo động từ Backend và chịu trách nhiệm phát cảnh báo khẩn cấp dưới dạng thông báo đẩy (Push Notification) đến thiết bị của người dùng dựa trên mã định danh `fcm_token`.

3.1.2 Các yêu cầu chức năng

Nhóm chức năng dành cho Người dùng

- **Cấu hình địa chỉ WiFi cho camera:** Khi lần đầu sử dụng thiết bị, hoặc reset cấu hình, người dùng có thể truy cập giao diện web do camera khởi tạo, để cấu hình địa chỉ WiFi cho camera.
- **Quản lý xác thực tài khoản:** Người dùng có thể đăng nhập, đăng xuất, tạo tài khoản mới, cũng như chỉnh sửa thông tin cá nhân.
- **Quản lý phòng:** Người dùng có quyền khởi tạo, chỉnh sửa thông tin hoặc xóa bỏ các phòng/khu vực cần giám sát (phòng khách, nhà kho, xưởng sản xuất...).
- **Quản lý camera:** Người dùng có thể thêm thiết bị camera mới vào hệ thống thông qua địa chỉ MAC của thiết bị, và gán camera đó vào một phòng cụ thể; cũng như điều chỉnh thông tin cũng như cấu hình ngưỡng báo động.
- **Giám sát trực tuyến:** Người dùng có thể giám sát theo dõi các camera cùng các cảm biến đi theo nó. Hệ thống phải cung cấp các thông tin: trạng thái camera, luồng video cũng như thông số cảm biến trực tiếp từ camera. Khi có sự cố khẩn cấp, người dùng phải nhận được cảnh báo từ hệ thống, và xem được hình ảnh hoặc video về đám cháy, từ đó thực hiện bấm nút xác nhận đã xử lý hiện trường an toàn.
- **Quản lý lịch sử cảnh báo:** Người dùng có thể xem các cảnh báo đã được phát đi từ trước đến nay, bao gồm cả hình ảnh và video.

Nhóm chức năng dành cho Camera

- **Cấu hình WiFi:** Camera có khả năng khởi tạo một web server tạm thời, cung cấp giao diện để người dùng có thể cấu hình địa chỉ cũng như mật khẩu WiFi. Trong môi trường thử nghiệm, do địa chỉ server không cố định, nên camera sẽ yêu cầu thêm địa chỉ server từ người dùng.
- **Tự động đồng bộ thông tin:** Camera cần phải định kỳ phát thông tin "giới thiệu bản thân" đến server. Server khi đó sẽ dựa vào thông tin địa chỉ MAC do người dùng cung cấp, tự động đồng bộ các thông tin còn lại cần thiết (model, danh sách cảm biến được hỗ trợ, ngưỡng cảm biến mặc định...)

- **Truyền phát video:** Camera cần phải phát luồng video stream liên tục cho server, để dịch vụ AI xử lý, đồng thời đẩy luồng video cho ứng dụng người dùng.
- **Gửi dữ liệu cảm biến:** Đọc dữ liệu cảm biến định kỳ, và gửi lại cho server để xử lý.
- **Phát cảnh báo:** Khi nhận được tín hiệu từ server, camera có nhiệm vụ phát tín hiệu cảnh báo tại chỗ bằng tín hiệu còi báo cũng như đèn LED.

Nhóm chức năng dành cho FCM

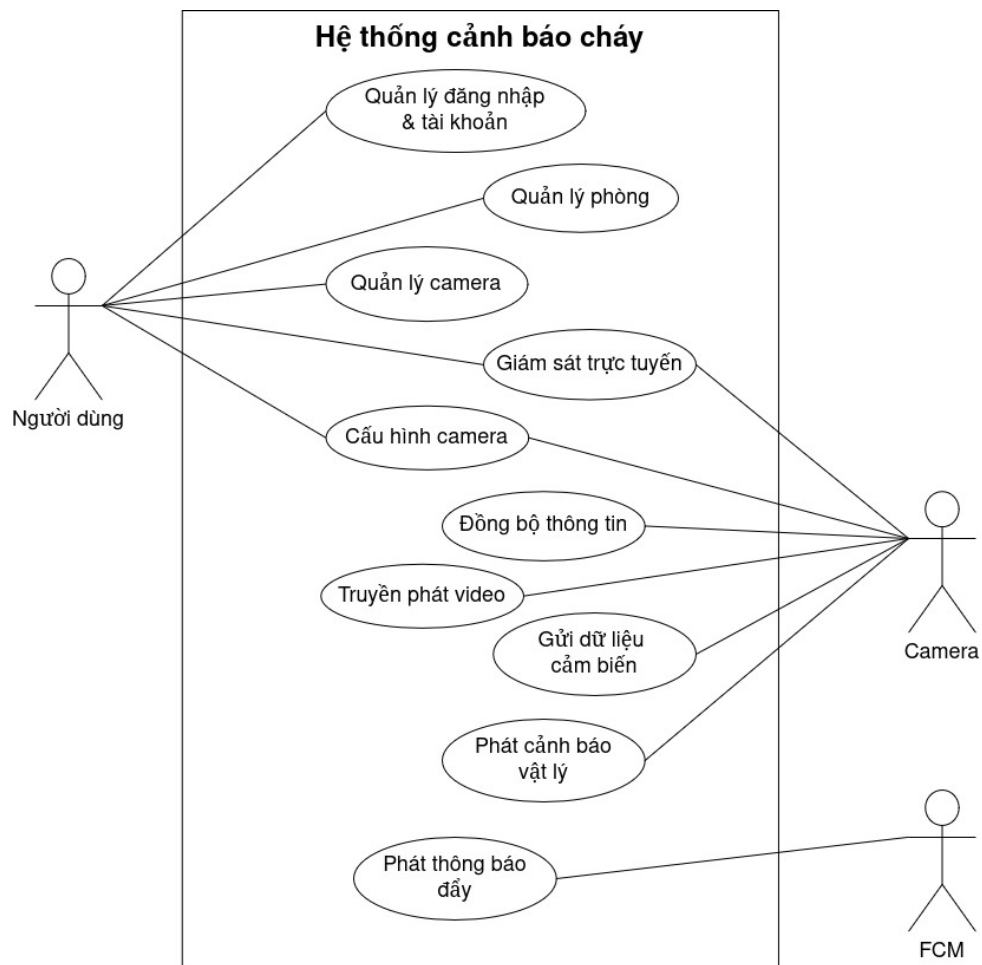
- **Phát thông báo đẩy:** Nhận dữ liệu cảnh báo khẩn cấp (tiêu đề, nội dung vụ cháy, mã phòng) được gửi từ server, và đẩy thông báo hiển thị ngay lập tức lên màn hình điện thoại di động của người dùng.

3.1.3 Các yêu cầu phi chức năng

Ngoài các yêu cầu chức năng kể trên, còn có một số yêu cầu phi chức năng như sau:

- **Hiệu năng và tốc độ xử lý:** Từ thời điểm camera phát hiện cháy đến khi thiết bị nhận cảnh báo, thời gian không vượt quá 5 giây. Camera đảm bảo được tốc độ khung hình 10 FPS, hạn chế tình trạng trễ giật khung hình.
- **Độ chính xác và tin cậy:** Hệ thống phải vận hành cơ chế kiểm tra chéo, kết hợp tất cả các thông tin cảm biến (bao gồm cả kết quả phân tích AI) trước khi kích hoạt báo động.
- **Bảo mật:** Áp dụng cơ chế xác thực JWT, mật khẩu người dùng phải được mã hóa, API phải được bảo vệ để ngăn chặn truy cập trái phép.

3.1.4 Biểu đồ use case tổng quan



Hình 3.1: Biểu đồ use case tổng quan

3.1.5 Đặc tả use case

Dưới đây là các use case của hệ thống

MÃ	UC01	TÊN	Quản lý đăng nhập & thông tin tài khoản
Tác nhân chính	Người dùng		
Mô tả	Người dùng đăng ký một tài khoản mới trên hệ thống, đăng nhập vào ứng dụng thông qua tên tài khoản và mật khẩu bảo mật, đăng xuất khi kết thúc phiên làm việc, và cập nhật thông tin hồ sơ cá nhân.		

Kích hoạt	Người dùng chưa đăng nhập, ấn vào nút đăng ký, đăng xuất hoặc quản lý tài khoản.
Luồng sự kiện chính	1. Người dùng truy cập hệ thống 2. Nếu chưa có tài khoản, người dùng truy cập giao diện đăng ký tài khoản mới 3. Người dùng điền thông tin đăng nhập hệ thống 4. Hệ thống trả kết quả thành công, đưa người dùng vào giao diện dashboard chính của hệ thống 5. Người dùng truy cập tính năng quản lý tài khoản, thay đổi thông tin nếu có 6. Người dùng nhấn vào nút Đăng xuất, kết thúc phiên làm việc
Luồng sự kiện con	2.1. Hệ thống hiển thị form đăng ký tài khoản, bao gồm username, họ tên, email, mật khẩu. 2.2. Người dùng điền thông tin, hệ thống xác nhận thành công. 5.1. Hệ thống hiển thị giao diện quản lý tài khoản, và các trường thông tin. 5.2. Người dùng điền thông tin, hệ thống xác nhận thành công.
Luồng ngoại lệ	2.1. Nếu thông tin đăng ký không hợp lệ, hệ thống báo lỗi 3.1. Nếu thông tin đăng nhập không đúng, hệ thống báo lỗi 5.1. Nếu thông tin chỉnh sửa không hợp lệ, hệ thống báo lỗi

Bảng 3.1: Đặc tả Use Case 01

MÃ	UC02	TÊN	Quản lý phòng
Tác nhân chính	Người dùng		
Mô tả	Người dùng tạo một phòng mới trên hệ thống, có thể chỉnh sửa các thông tin của phòng hiện có.		
Kích hoạt	Người dùng truy cập tính năng "Quản lý phòng" trên giao diện hệ thống.		

Luồng sự kiện chính	1. Người dùng truy cập tính năng "Quản lý phòng" 2. Hệ thống hiển thị danh sách phòng hiện có 3. Người dùng tạo mới phòng 4. Người dùng chọn và chỉnh sửa phòng hiện có 5. Người dùng xóa phòng hiện có
Luồng sự kiện con	3.1. Hệ thống hiển thị form tạo phòng mới. 3.2. Người dùng điền thông tin, hệ thống trả về kết quả. 4.1. Hệ thống hiển thị form chỉnh sửa phòng hiện tại. 4.2. Người dùng cập nhật thông tin, hệ thống trả về kết quả. 5.1. Hệ thống hiện pop up yêu cầu xác nhận xóa phòng. 5.2. Người dùng xác nhận, hệ thống trả về kết quả.

Bảng 3.2: Đặc tả Use Case 02

MÃ	UC03	TÊN	Quản lý camera
Tác nhân chính	Người dùng		
Mô tả	Người dùng đăng ký camera mới, có thể chỉnh sửa các thông tin của camera hiện có.		
Kích hoạt	Người dùng truy cập tính năng "Quản lý camera" trên giao diện hệ thống.		
Luồng sự kiện chính	1. Người dùng truy cập tính năng "Quản lý camera" 2. Hệ thống hiển thị danh sách phòng hiện có 3. Người dùng chọn "Thêm camera" 4. Người dùng chọn và chỉnh sửa camera hiện có 5. Người dùng xóa camera hiện có		

Luồng sự kiện con	3.1. Hệ thống hiển thị form thêm camera mới. 3.2. Người dùng điền thông tin, hệ thống trả về kết quả. 3.3. Hệ thống đồng bộ thông tin camera, rồi trả về trạng thái mới nhất của camera 4.1. Hệ thống hiển thị form chỉnh sửa camera hiện tại, bao gồm các ngưỡng cấu hình của cảm biến. 4.2. Người dùng cập nhật thông tin, hệ thống trả về kết quả. 5.1. Hệ thống hiện pop up yêu cầu xác nhận xóa camera. 5.2. Người dùng xác nhận, hệ thống trả về kết quả.
Luồng ngoại lệ	3.1. Nếu camera đã được đăng ký, hệ thống báo lỗi 4.1. Nếu thông tin chỉnh sửa không hợp lệ, hệ thống báo lỗi

Bảng 3.3: *Đặc tả Use Case 03*

Mã	UC04	Tên	Giám sát trực tuyến
Tác nhân chính	Người dùng, Camera		
Mô tả	Cho phép người dùng theo dõi luồng video trực tiếp, trạng thái kết nối và các thông số cảm biến môi trường thời gian thực từ camera. Khi hệ thống phân tích phát hiện sự cố cháy, hệ thống sẽ đẩy cảnh báo khẩn cấp để người dùng kiểm tra bằng chứng (ảnh/video đám cháy) và bấm nút xác nhận đã xử lý an toàn tại hiện trường.		
Kích hoạt	Người dùng truy cập vào chức năng "Giám sát trực tuyến" trên giao diện hệ thống.		

Luồng sự kiện chính	1. Người dùng chọn một khu vực phòng cần theo dõi trên màn hình điều khiển. 2. Hệ thống server thiết lập kết nối đến client và hiển thị danh sách các camera thuộc phòng đó kèm trạng thái của camera (online/offline/chưa kết nối). 3. Hệ thống tải và phát luồng video trực tuyến (stream_url) từ camera lên màn hình người dùng. 4. Camera liên tục đọc các chân tín hiệu vật lý và gửi dữ liệu cảm biến vật lý về server. 5. Máy chủ tiếp nhận dữ liệu cảm biến, gửi liên tục lên client để cập nhật liên tục giá trị cảm biến trên giao diện người dùng. 6. Người dùng kết thúc phiên giám sát bằng cách thoát khỏi màn hình chức năng.
Luồng sự kiện con	(4) Phát cảnh báo cháy: 4.1. Trong quá trình giám sát, tiến trình phân tích AI liên tục quét luồng video, nhận diện thấy lửa vượt ngưỡng tin cậy và phát tín hiệu cảnh báo, đồng thời lưu ảnh/video vào bộ nhớ tạm. 4.2. Server tiến hành kiểm tra, kết hợp với thông số các cảm biến vật lý do camera gửi để xác nhận đám cháy xảy ra. 4.3. Server lập tức phát tín hiệu tới camera để phát tín hiệu còi báo. 4.4. Server tiến hành tạo bản ghi cảnh báo rồi vào cơ sở dữ liệu. 4.5. Server gửi cảnh báo lên client qua kết nối đã thiết lập, đồng thời gọi FCM để phát cảnh báo đẩy tới các thiết bị của người dùng đó. 4.6. Server gửi lệnh tới tiến trình AI đang nắm giữ ảnh/video tạm, đưa chúng vào bộ nhớ chính và cập nhật vào cơ sở dữ liệu. 4.7. Người dùng nhận cảnh báo từ hệ thống. 4.8. Sau khi đám cháy được dập tắt, người dùng nhấn xác nhận "Đã xử lý", hệ thống tiến hành cập nhật.

Luồng ngoại lệ	3. Nếu camera bị mất kết nối đột ngột: - MQTT Broker sẽ nhận diện qua cơ chế Keep-Alive/Last-Will, gửi thông tin tới server thông báo rằng camera đã mất kết nối - Server sẽ cập nhật trạng thái tại cơ sở dữ liệu thành "Offline" - Client khi nhận được thông tin, sẽ hiển thị giao diện báo "Thiết bị ngoại tuyến" 4. Nếu AI phát hiện cháy, nhưng cảm biến chưa đủ ngưỡng kích hoạt: - Hệ thống coi đây là báo động giả, không tiến hành lưu và phát cảnh báo. - Sau một khoảng thời gian, dữ liệu ảnh/video tạm từ server sẽ được xóa bỏ vì không còn được dùng.
-----------------------	--

Bảng 3.4: Đặc tả Use Case 04

Mã	UC05	Tên	Tự động đồng bộ thông tin
Tác nhân chính	Camera		
Mô tả	Camera phát tín hiệu chứa địa chỉ MAC lên máy chủ để hệ thống tự động nhận diện và cập nhật các thông số kỹ thuật, cấu hình ngưỡng tương thích mặc định.		
Kích hoạt	Camera thiết lập kết nối thành công tới mạng WiFi và kết nối tới MQTT broker của hệ thống		
Luồng sự kiện chính	1. Camera tự động đóng gói thông tin của mình, bao gồm địa chỉ MAC vật lý, loại camera, và các cảm biến có hỗ trợ vào bản tin thông điệp khởi động. 2. Camera xuất bản thông điệp này lên một topic cấu hình chung qua MQTT. 3. Server tiếp nhận bản tin, kiểm tra mã MAC và lấy ra các thông tin thiết bị. 4. Máy chủ tự động ghi nhận trạng thái trực tuyến (Online) và cập nhật đồng bộ các tham số ngưỡng mặc định cho thiết bị trong CSDL.		

Luồng sự kiện con	Không có
Luồng ngoại lệ	4.1. Địa chỉ MAC chưa được người dùng đăng ký trước Nếu Camera gửi địa chỉ MAC lên nhưng kiểm tra cơ sở dữ liệu thấy mã này chưa từng được khai báo, server từ chối đồng bộ.

Bảng 3.5: Đặc tả Use Case 05

MÃ	UC06	TÊN	Truyền phát video
Tác nhân chính	Camera		
Mô tả	Camera liên tục đẩy luồng video trực tiếp từ cảm biến ống kính về máy chủ ứng dụng để phục vụ xử lý nhận diện đám cháy AI và luồng xem của người dùng.		
Kích hoạt	Thiết bị chuyển sang trạng thái hoạt động bình thường sau khi đồng bộ kết nối.		
Luồng sự kiện chính	1. Cảm biến hình ảnh trên camera bắt đầu thu nhận các khung hình số liên tục. 2. Mạch mã hóa dữ liệu nén luồng hình ảnh thành định dạng stream chuẩn. 3. Camera thực hiện truyền phát luồng dữ liệu video này liên tục theo địa chỉ URL cấu hình về phía địa chỉ IP máy chủ. 4. Luồng dữ liệu được chia tách phục vụ đồng thời cho dịch vụ AI xử lý và client xem trực tiếp.		
Luồng sự kiện con	Không có		
Luồng ngoại lệ	Không có		

Bảng 3.6: Đặc tả Use Case 06

MÃ	UC07	TÊN	Gửi dữ liệu cảm biến
Tác nhân chính	Camera		

Mô tả	Thiết bị tự động đọc giá trị số từ các chân linh kiện cảm biến vật lý theo chu kỳ định sẵn và gửi dữ liệu môi trường thô về máy chủ qua MQTT
Kích hoạt	Chu kỳ thời gian lập trình trên bo mạch vi điều khiển (ví dụ: định kỳ mỗi 1 giây/lần)
Luồng sự kiện chính	1. Mạch điều khiển đọc tín hiệu từ các cảm biến. 2. Dữ liệu được đóng gói thành chuỗi JSON định dạng chuẩn. 3. Camera thực hiện gửi xuất bản (Publish) gói tin này lên chủ đề riêng của thiết bị về MQTT broker.
Luồng sự kiện con	Không có
Luồng ngoại lệ	Không có

Bảng 3.7: Đặc tả Use Case 07

MÃ	UC08	TÊN	Phát cảnh báo tại chỗ
Tác nhân chính	Camera		
Mô tả	Thiết bị nhận lệnh từ server gửi xuống để lập tức kích hoạt còi hú âm lượng cao và hệ thống đèn LED phát báo động trực tiếp tại vị trí xảy ra sự cố cháy.		
Kích hoạt	Camera nhận được bản tin lệnh MQTT báo động do server phát xuống khi xác định có cháy.		
Luồng sự kiện chính	1. Bo mạch Camera nhận và giải mã bản tin lệnh điều khiển khẩn cấp. 2. Vi điều khiển xuất tín hiệu điện áp mức cao ra chân điều khiển linh kiện ngoại vi. 3. Mạch còi điện tử kích hoạt tiếng hú còi chu kỳ liên tục. 4. Hệ thống đèn LED chuyển chế độ nhấp nháy tần suất cao cảnh báo. 5. Thiết bị duy trì trạng thái báo động vật lý cho đến khi nhận được bản tin lệnh tắt còi từ máy chủ (khi người dùng bấm nút xác nhận an toàn).		
Luồng sự kiện con	Không có		

Luồng ngoại lệ	Không có
-----------------------	----------

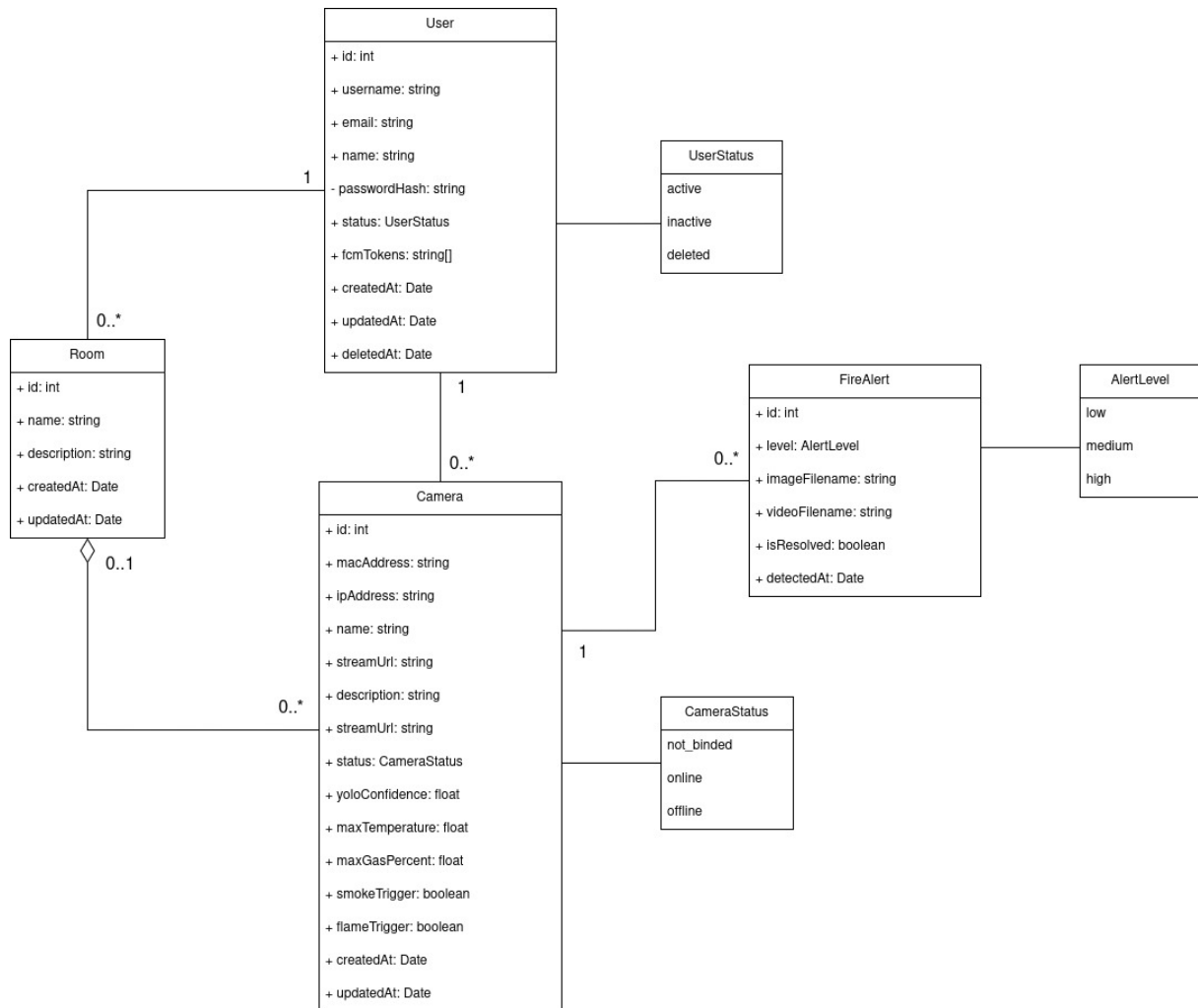
Bảng 3.8: *Đặc tả Use Case 08*

MÃ	UC09	TÊN	Phát thông báo đẩy lên thiết bị người dùng
Tác nhân chính	Hệ thống FCM (Firebase Cloud Messaging)		
Mô tả	Dịch vụ FCM tiếp nhận payload thông tin hỏa hoạn và mã thiết bị từ server, thực hiện định tuyến mạng và ép hiển thị tin nhắn cảnh báo khẩn cấp lên điện thoại người sở hữu		
Kích hoạt	Server gọi API dịch vụ của FCM để truyền thông số sự cố cháy khẩn cấp vừa được khởi tạo.		
Luồng sự kiện chính	1. Cổng kết nối FCM tiếp nhận yêu cầu đẩy dữ liệu chứa: Tiêu đề, nội dung cảnh báo vị trí phòng hỏa hoạn và token thiết bị mục tiêu. 2. FCM thực hiện kiểm tra trạng thái token và xác định địa chỉ thiết bị trên mạng internet toàn cầu. 3. FCM định tuyến luồng tin nhắn khẩn cấp qua các kênh truyền dẫn dẫn không dây. 4. Thiết bị di động nhận gói tin, hệ điều hành ép mở và hiển thị pop-up thông báo nhấp nháy cùng âm thanh cảnh báo đặc biệt trên màn hình người dùng.		
Luồng sự kiện con	Không có		
Luồng ngoại lệ	2.1. Nếu token thiết bị không còn hoạt động, FCM trả về mã lỗi BadDeviceToken cho máy chủ ứng dụng để hệ thống tự động xóa bỏ mã lỗi này ra khỏi cơ sở dữ liệu bảng users.		

Bảng 3.9: *Đặc tả Use Case 09*

3.2 Mô hình hóa cấu trúc

Để mô hình hóa được cấu trúc của hệ thống, dưới đây là sơ đồ lớp lĩnh vực của hệ thống cảnh báo cháy:



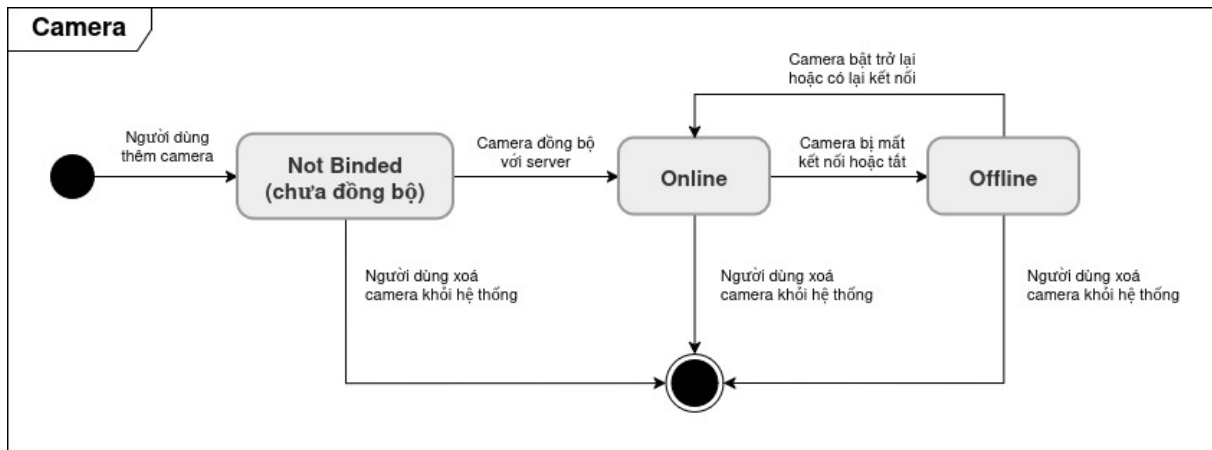
Hình 3.2: Sơ đồ lớp của hệ thống

Trọng tâm của sơ đồ là thực thể User, đóng vai trò là chủ sở hữu hệ thống. Một phòng có thể quản lý nhiều thiết bị Camera nhưng thiết bị vẫn tồn tại độc lập trong cơ sở dữ liệu nếu phòng bị xóa bỏ. Lớp Camera là nơi lưu trữ các cấu hình thiết yếu bao gồm các thông tin cơ bản, và các ngưỡng cảm biến an toàn vật lý. Khi có hỏa hoạn xảy ra, lớp thiết bị này sẽ sinh ra các bản ghi sự kiện tại lớp Fire_Alert, hỗ trợ hệ thống lưu vết mốc thời gian cùng các tệp tin hình ảnh, video bằng chứng.

3.3 Mô hình hóa hành vi

3.3.1 Sơ đồ máy trạng thái cho đối tượng camera

Dưới đây là sơ đồ máy trạng thái dành cho đối tượng camera, nhằm mục đích mô tả một cách trực quan và toàn diện vòng đời hoạt động của đối tượng camera trong hệ thống.

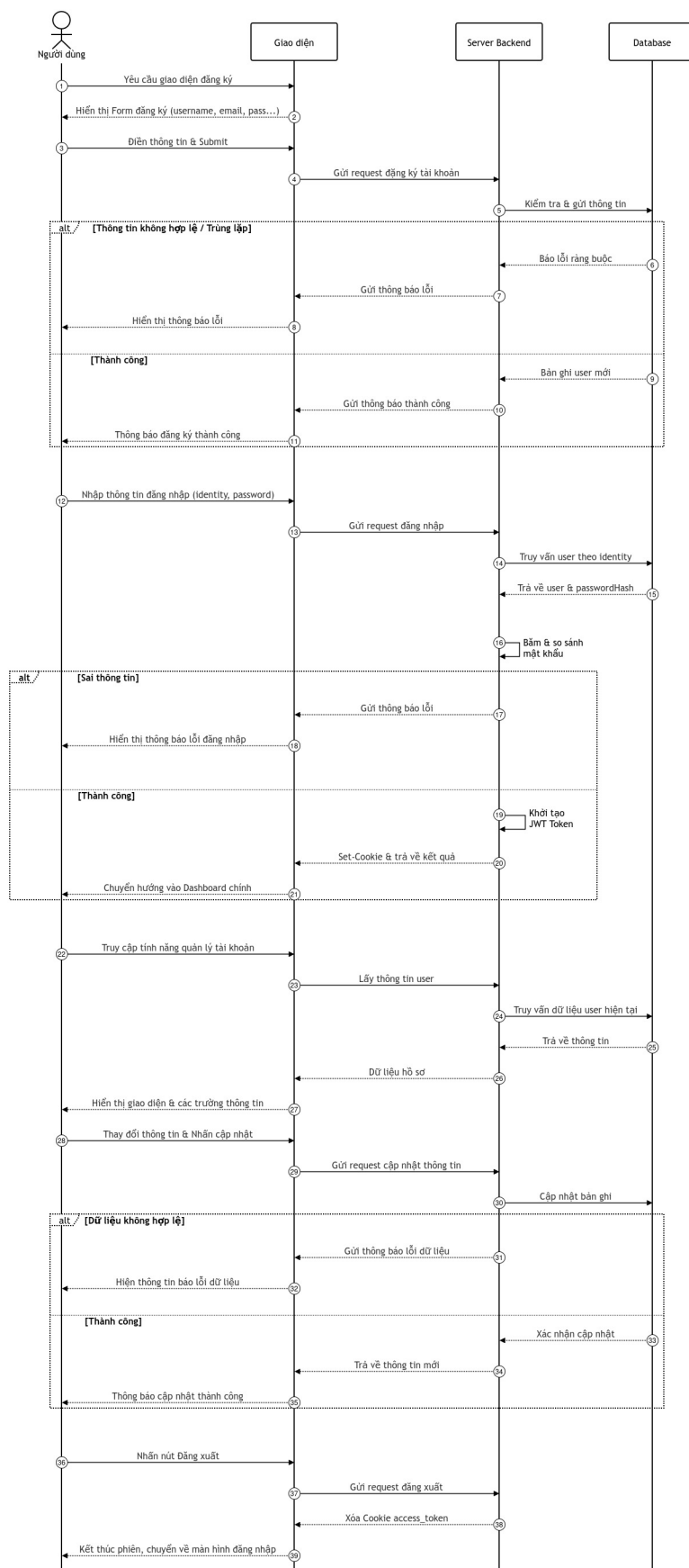


Hình 3.3: Sơ đồ máy trạng thái cho đối tượng Camera

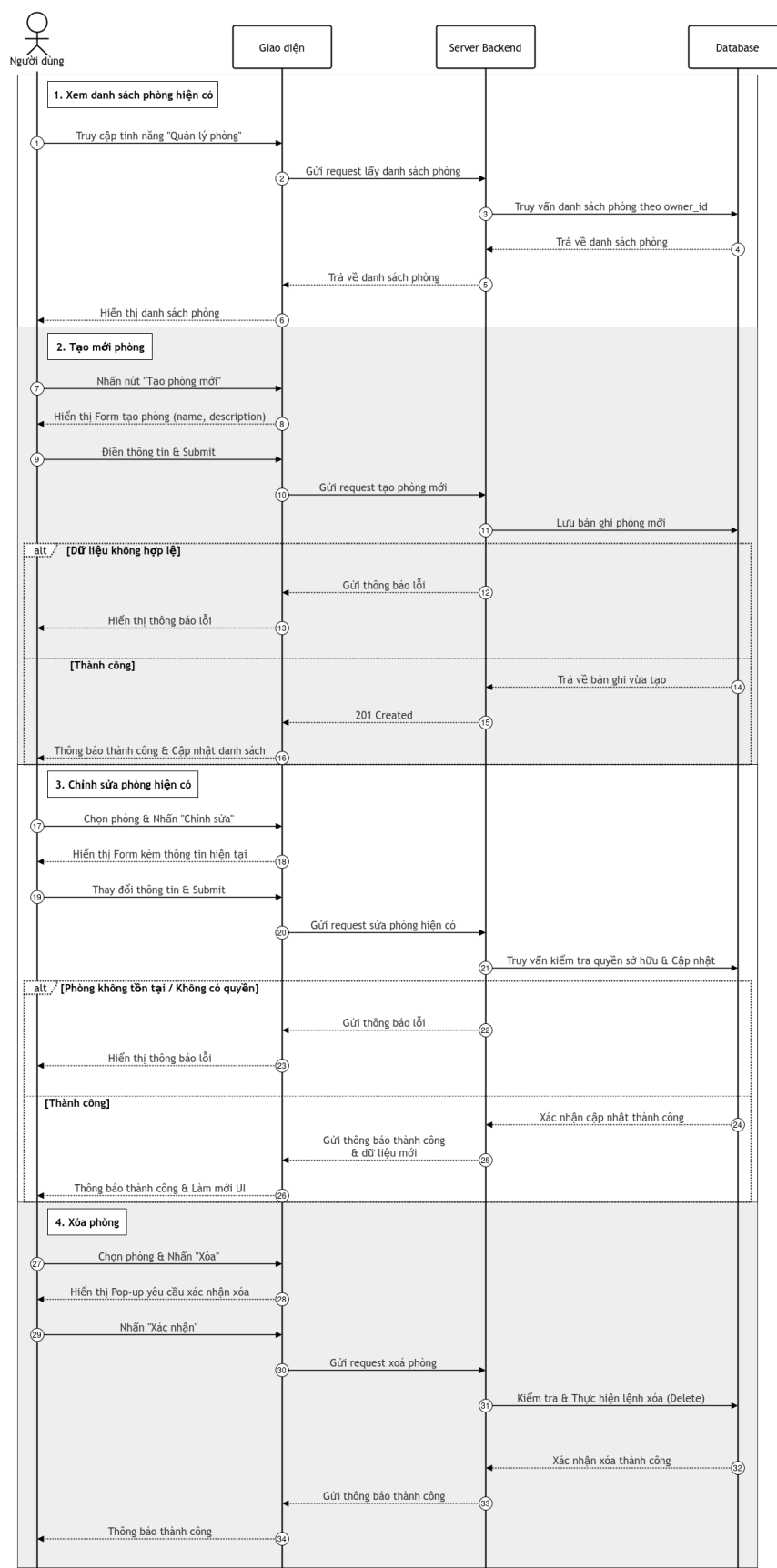
Trạng thái của camera được xoay quanh ba trạng thái chính, cụ thể là Not Binded (Chưa đồng bộ), Online (Trực tuyến) và Offline (Ngoại tuyến). Khởi đầu, người dùng thêm camera thì server sẽ ghi nhận camera ở trạng thái Not Binded. Trong khoảng thời gian đó, camera sẽ liên tục gửi bản tin giới thiệu về mình qua giao thức MQTT tới server. Khi server nhận được, các thông tin còn lại được đồng bộ, và trạng thái camera chuyển về Online. Camera sẽ hoạt động liên tục, đến khi mất kết nối hoặc mất nguồn thì trạng thái sẽ chuyển về Offline, và chuyển về Online ngay lập tức khi có kết nối hoặc có nguồn trở lại. Tại mọi trạng thái, người dùng có thể xóa đối tượng camera khỏi hệ thống.

3.3.2 Sơ đồ tuần tự cho các use case

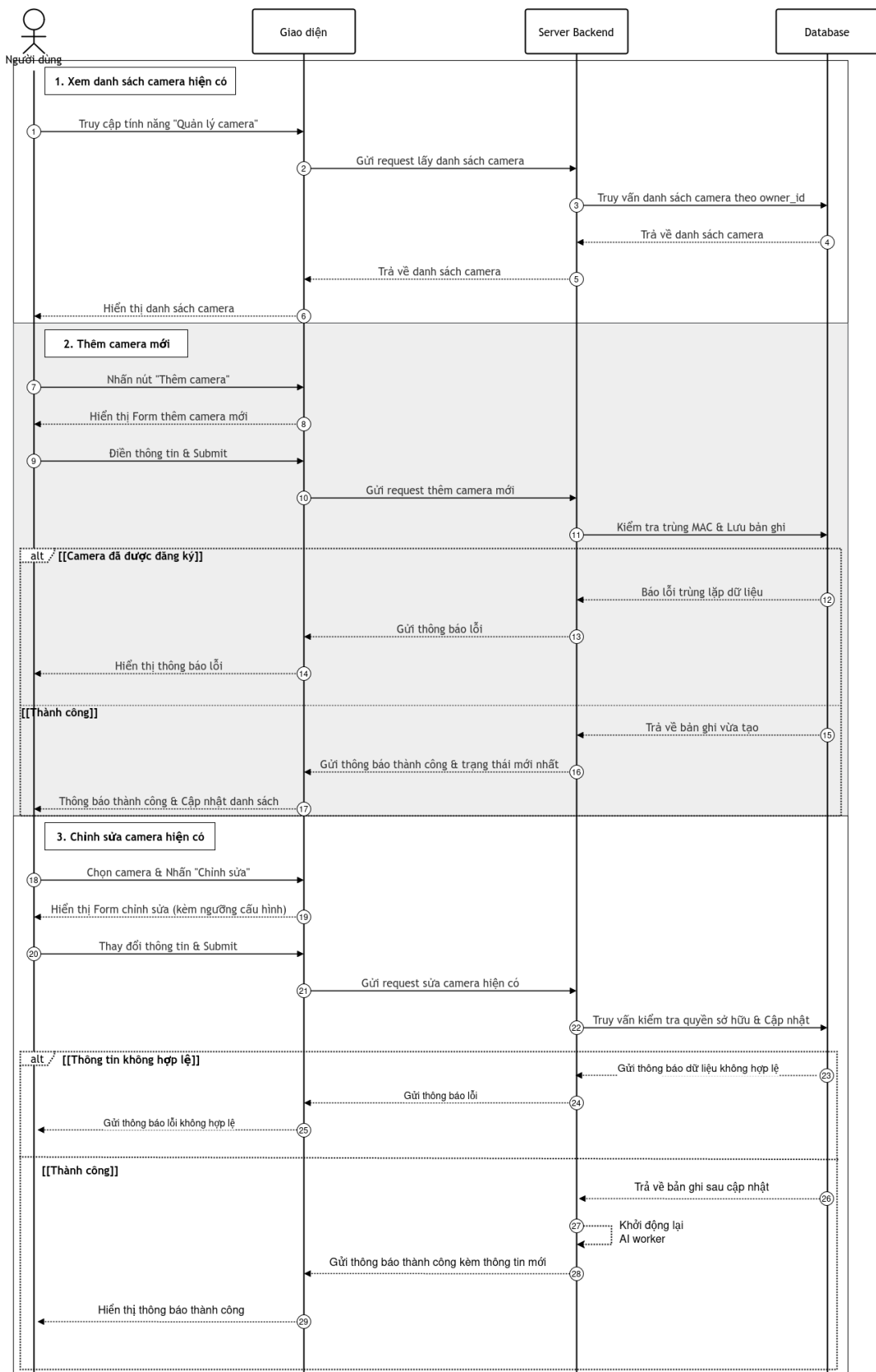
Để thể hiện luồng hoạt động chi tiết, cũng như sự phối hợp giữa các thành phần trong hệ thống ở từng use case, dưới đây là các sơ đồ tuần tự dành cho từng use case.



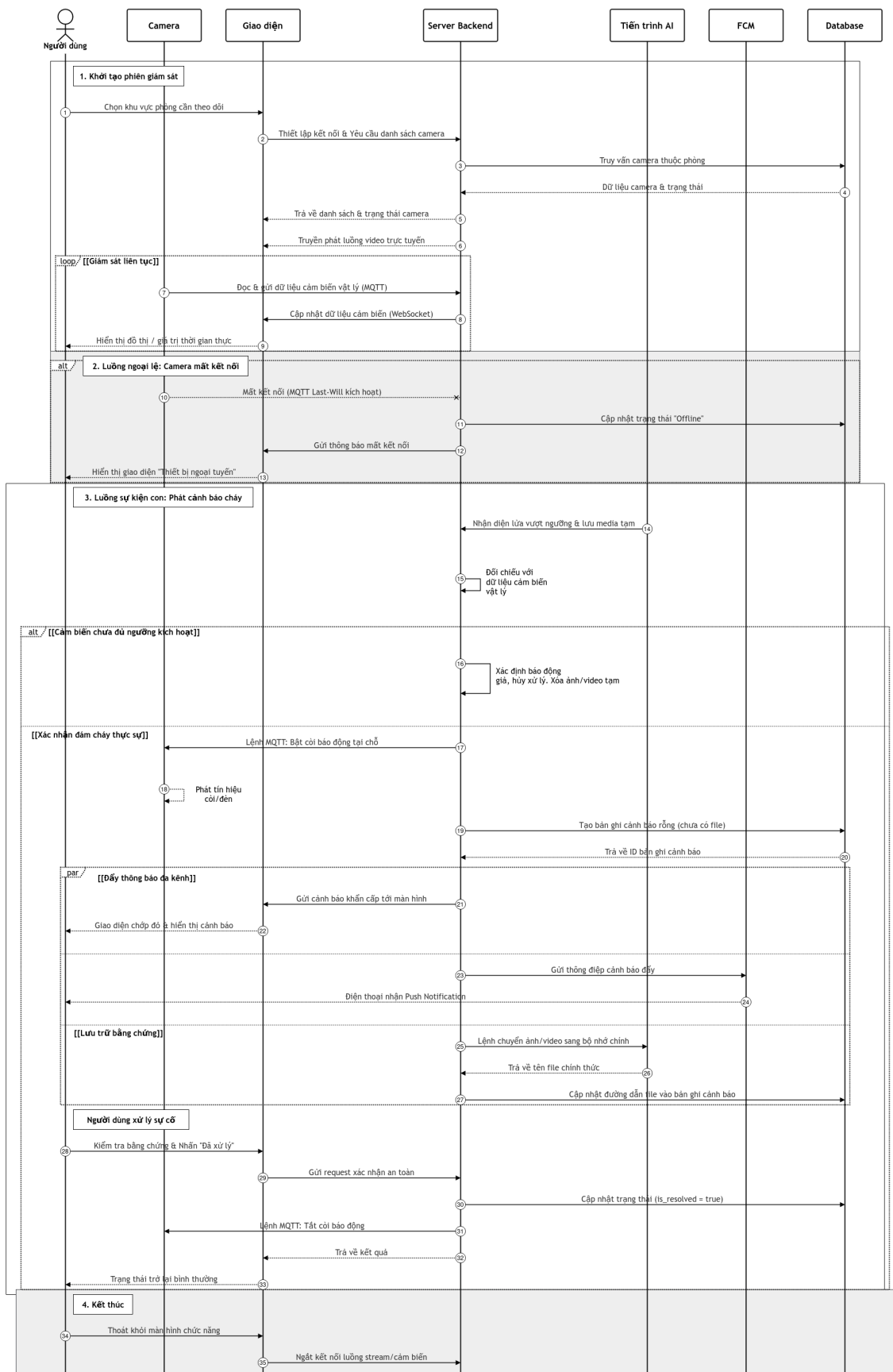
Hình 3.4: Sơ đồ tuần tự cho Use Case 1



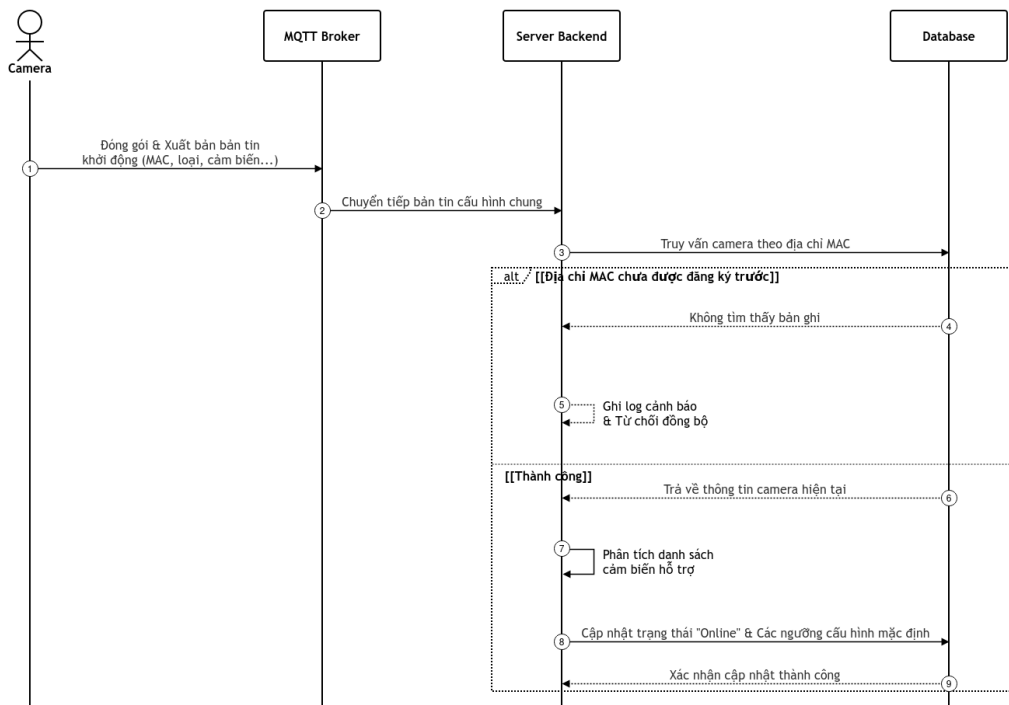
Hình 3.5: Sơ đồ tuần tự cho Use Case 2



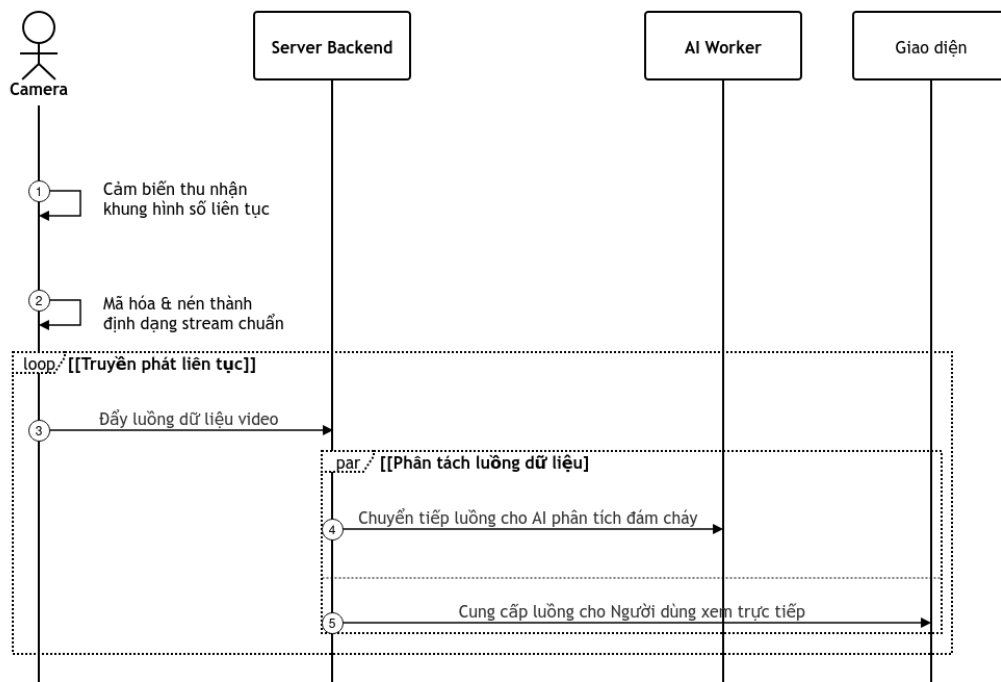
Hình 3.6: Sơ đồ tuần tự cho Use Case 3



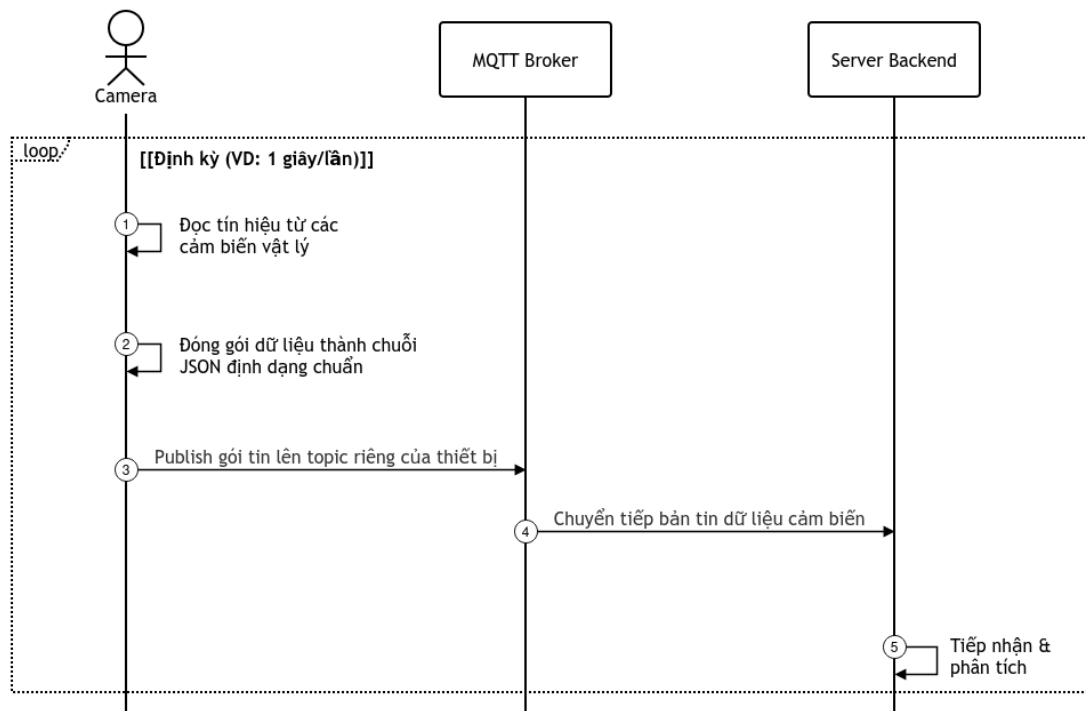
Hình 3.7: Sơ đồ tuần tự cho Use Case 4



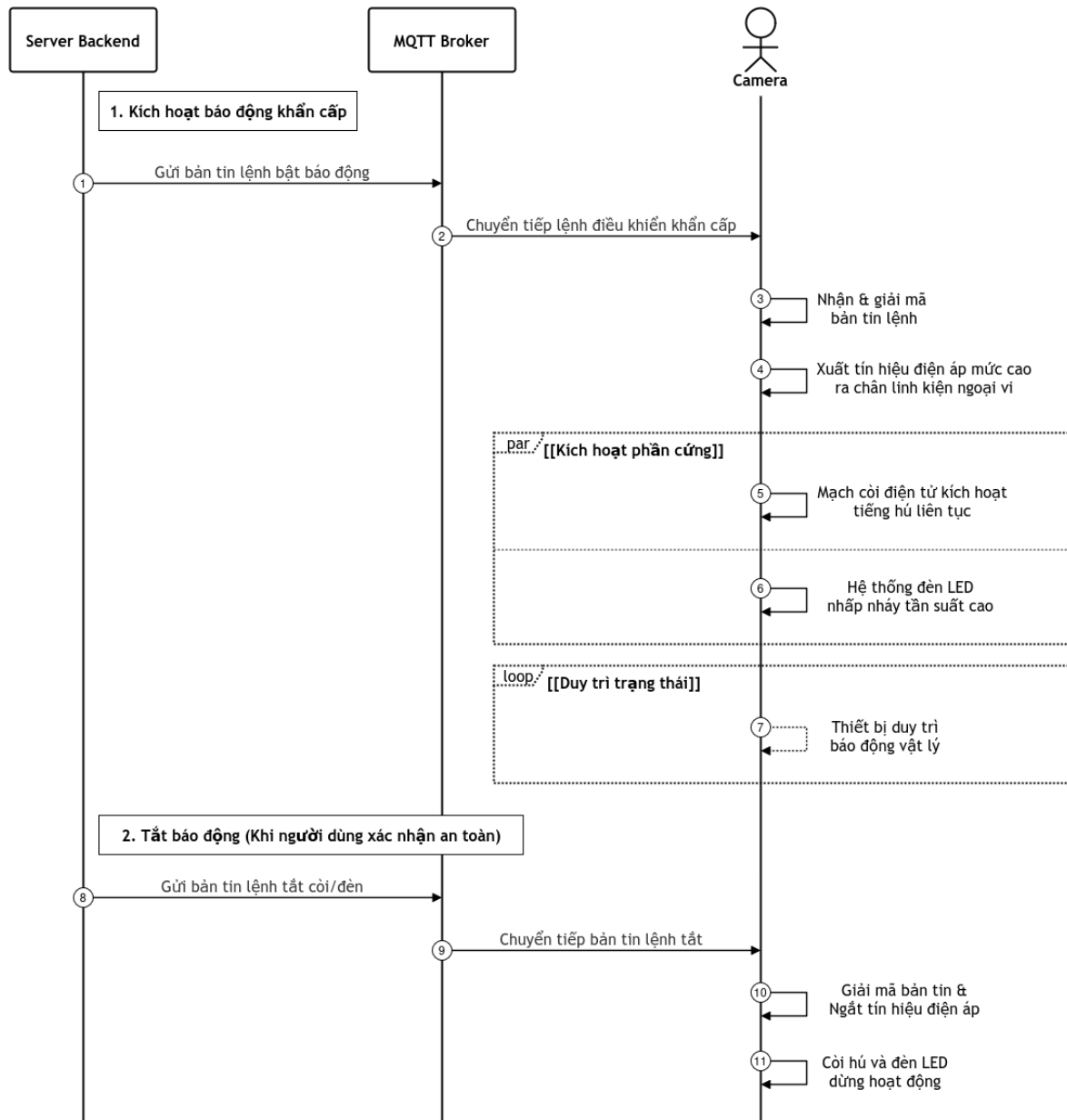
Hình 3.8: Sơ đồ tuần tự cho Use Case 5



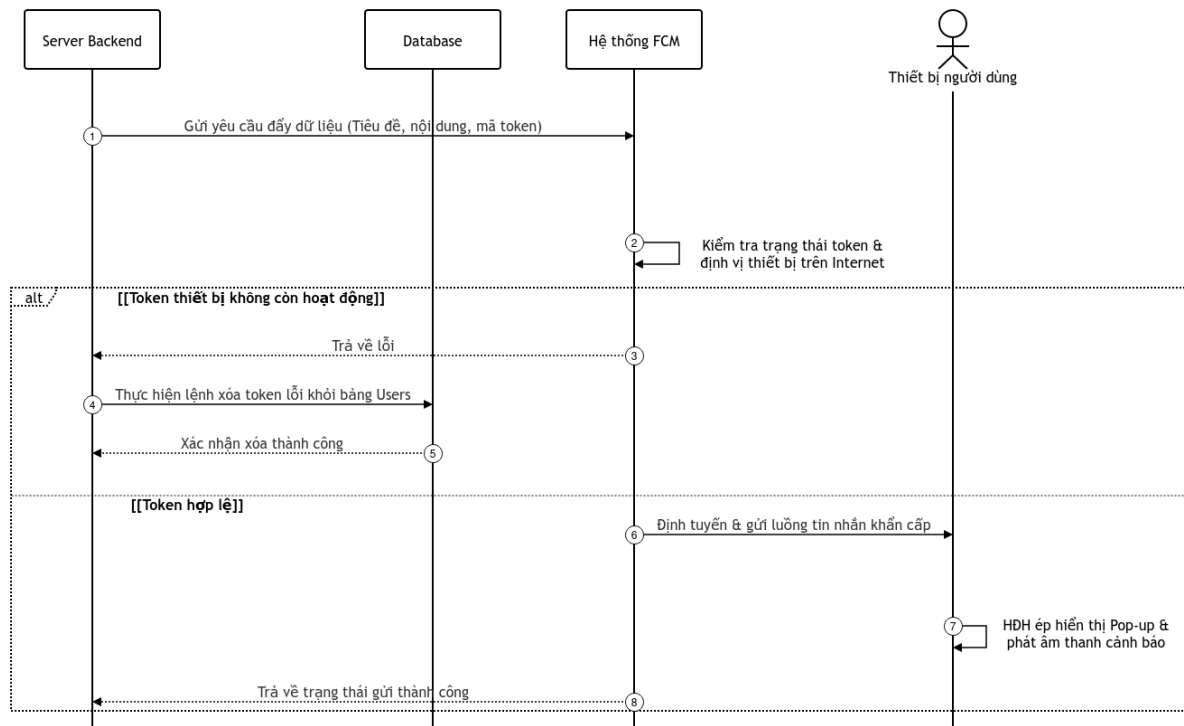
Hình 3.9: Sơ đồ tuần tự cho Use Case 6



Hình 3.10: Sơ đồ tuần tự cho Use Case 7



Hình 3.11: Sơ đồ tuần tự cho Use Case 8

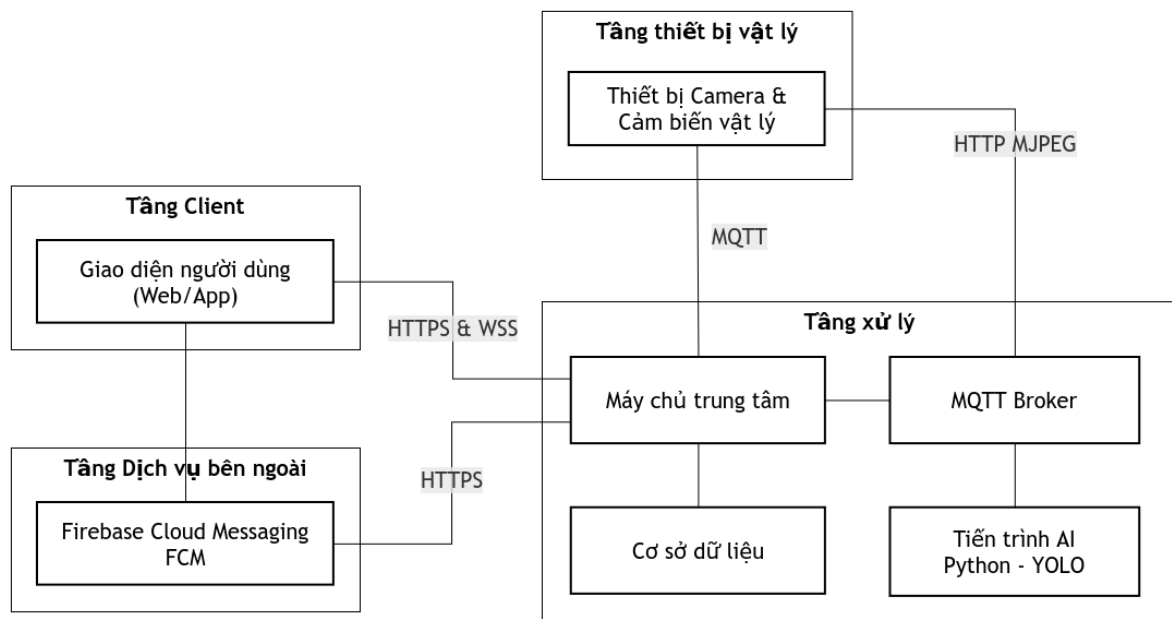


Hình 3.12: Sơ đồ tuần tự cho Use Case 9

CHƯƠNG 4: THIẾT KẾ HỆ THỐNG

4.1 Thiết kế kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình phân tán, với 4 tầng cốt lõi tương tác với nhau trong thời gian thực.



Hình 4.1: Sơ đồ kiến trúc hệ thống

Tầng thiết bị vật lý bao gồm các camera tích hợp cảm biến môi trường (khói, lửa, nhiệt độ, khí gas), được kết nối với vi điều khiển ESP32. Thành phần này chịu trách nhiệm thu thập dữ liệu thô của cảm biến tại hiện trường, đóng gói gửi cho server; truyền phát luồng video liên tục; và phát cảnh báo.

Tầng xử lý là tầng quan trọng nhất trong hệ thống, bao gồm 4 thành phần được triển khai đồng bộ trên một máy chủ nhằm tối ưu chi phí vận hành. Trong đó, máy chủ trung tâm đóng vai trò hạt nhân điều phối, chịu trách nhiệm xử lý logic nghiệp vụ, xác thực người dùng, quản lý quyền hạn và điều khiển các tiến trình con. MQTT Broker đóng vai trò trạm trung chuyển các bản tin, duy trì kết nối trực tiếp với các Camera, đóng vai trò cầu nối giao tiếp giữa máy chủ trung tâm tới các Camera cũng như các tiến trình xử lý AI. Cơ sở dữ liệu đóng vai trò lưu trữ dữ

liệu có cấu trúc bao gồm thông tin người dùng, cấu hình phòng, thiết bị và lịch sử cảnh báo. Cuối cùng là các tiến trình AI (AI Worker) chạy ngầm độc lập do máy chủ trung tâm quản lý, mỗi tiến trình đảm nhận nhiệm vụ xử lý luồng video cho mỗi camera, bao gồm phân tích hình ảnh dựa trên mô hình YOLO để đưa ra cảnh báo, cũng như lưu lại hình ảnh và video khi xảy ra hỏa hoạn.

Tầng client đóng vai trò kết nối trực tiếp giữa người dùng cuối và hệ thống, với thành phần chủ chốt chính là ứng dụng di động hoặc giao diện Web dành cho người dùng cuối. Nhiệm vụ của thành phần này là phát luồng video trực tiếp và thông tin cảm biến theo thời gian thực, phát cảnh báo khi có cháy xảy ra, cũng như cho phép người dùng cấu hình hệ thống.

Ngoài hệ thống còn có tầng dịch vụ bên ngoài với hạ tầng Firebase Cloud Messaging (FCM) của Google, có nhiệm vụ tiếp nhận yêu cầu đẩy dữ liệu khẩn cấp từ máy chủ trung tâm và định tuyến tin nhắn đến ứng dụng người dùng để tiến hành phát cảnh báo đẩy khẩn cấp khi có cháy.

4.2 Thiết kế giao tiếp

Hệ thống sử dụng mô hình giao tiếp đa giao thức để tối ưu hóa việc truyền tải dữ liệu. Trong đó, RESTful API đảm nhận các tác vụ quản trị, WebSocket truyền dữ liệu thời gian thực lên ứng dụng, và MQTT chịu trách nhiệm giao tiếp hiệu năng cao với thiết bị phần cứng cũng như các tiến trình AI.

4.2.1 Đặc tả các RESTful API

Phương thức	Endpoint	Quyền truy cập	Mô tả chức năng
POST	/auth/register	Public	Đăng ký tài khoản người dùng mới.
POST	/auth/login	Public	Đăng nhập, xác thực và nhận JWT Token / Cookie.
POST	/auth/logout	Người dùng	Đăng xuất, hủy phiên làm việc.
GET	/rooms	Người dùng	Lấy danh sách các phòng thuộc sở hữu.

POST	/rooms	Người dùng	Khởi tạo một phòng giám sát mới.
PUT	/rooms/:id	Người dùng	Cập nhật thông tin phòng (tên, mô tả).
DELETE	/rooms/:id	Người dùng	Xóa phòng ra khỏi hệ thống.
GET	/cameras	Người dùng	Lấy danh sách camera của người dùng.
POST	/cameras	Người dùng	Đăng ký một camera mới vào phòng.
PUT	/cameras/:id	Người dùng	Cập nhật cấu hình ngưỡng cảm biến của camera.
DELETE	/cameras/:id	Người dùng	Xóa camera khỏi hệ thống.
PUT	/users/fcm-token	Người dùng	Cập nhật hoặc thêm mới FCM Token cho thiết bị di động.
GET	/alerts	Người dùng	Lấy danh sách các cảnh báo cháy của người dùng.
GET	/alerts/camera/:id	Người dùng	Lấy danh sách các cảnh báo cháy theo camera.
PUT	/alerts/handle/:id	Người dùng	Đánh dấu đã xử lý cảnh báo cho 1 cảnh báo.
PUT	/alerts/handle-all	Người dùng	Đánh dấu đã xử lý tất cả các cảnh báo.

Bảng 4.1: *Danh sách các Endpoint RESTful API*

4.2.2 Đặc tả giao tiếp WebSocket

Kênh WebSocket Secure (WSS) được thiết lập nhằm duy trì kết nối bất đồng bộ, liên tục giữa server và client để đẩy dữ liệu mà không cần tải lại trang. Khi người dùng truy cập màn hình "Giám sát trực tuyến", client sẽ thiết lập một kết nối WebSocket tới Gateway của server.

Tên sự kiện (Event Name)	Định dạng Payload (JSON)	Ý nghĩa và Nghiệp vụ kích hoạt
VIDEO_FRAME _{camera_id}	Chuỗi Base64 thô (string)	Ý nghĩa: Đẩy luồng khung hình video trực tiếp. Kích hoạt: AI Worker lấy luồng video từ camera, gửi qua MQTT và được server phát thẳng lên giao diện để người dùng xem trực tiếp.
SENSOR_DATA _{mac_address}	{ "temperature": float, "gasPercent": float, "smoke": bool, "flame": bool }	Ý nghĩa: Cập nhật thông số cảm biến thời gian thực. Kích hoạt: Camera gửi bản tin qua MQTT, server tiếp nhận và đẩy ngay sang WebSocket để hiển thị realtime trên client.
FIRE_ALARM	{ "id": int, "camera_id": int, "timestamp": string, "message": string, "trigger_source": string }	Ý nghĩa: Cảnh báo hỏa hoạn khẩn cấp lập tức. Kích hoạt: Kích hoạt ngay khi server phát hiện đám cháy thực sự. Sự kiện được bắn xuống tức thời mà không cần chờ tệp tin hình ảnh/video bằng chứng khởi tạo xong.
ALERTS_REFRESH	{ "camera_id": int }	Ý nghĩa: Lệnh yêu cầu đồng bộ/tải lại danh sách lịch sử. Kích hoạt: Phát ra khi tiến trình AI (hoặc tác vụ Fallback của hệ thống) đã xử lý xong tệp ảnh/video bằng chứng đám cháy, thông báo cho client cập nhật giao diện để hiển thị bằng chứng.

DEVICE_ STATUS_ CHANGED	{ "camera_id": int, "mac": string, "status": "online" "offline" }	Ý nghĩa: Cập nhật trạng thái kết nối phần cứng. Kích hoạt: Khi Camera thay đổi trạng thái hoạt động (báo kết nối thành công hoặc bị broker quét mất tín hiệu qua cơ chế Last-Will), giúp giao diện đổi màu trạng thái thiết bị lập tức.
-------------------------------	---	--

Bảng 4.2: Danh mục đặc tả các sự kiện truyền phát qua WebSocket

4.2.3 Đặc tả giao tiếp MQTT

Cấu trúc Topic	Bên gửi (Pub)	Bên nhận (Sub)	Cấu trúc Payload và Ý nghĩa
discovery/ cameras	Camera	Server	Payload: JSON chứa MAC, IP, streamUrl. Ý nghĩa: Camera tự động khai báo cấu hình phần cứng và báo trực tuyến khi khởi động.
sensors/ {mac_address}	Camera	Server	Payload: {"temp": float, "gas": float, "smoke": bool, "flame": bool} Ý nghĩa: Camera đẩy dữ liệu thô môi trường định kỳ.
commands/ {mac_address}	Server	Camera	Payload: {"buzzer": 0 1, "led": 0 1} Ý nghĩa: Máy chủ phát lệnh điều khiển khẩn cấp bật/tắt còi hú và đèn LED tại chỗ.
status/ {mac_address}	MQTT Broker	Server	Payload: Chuỗi "offline" Ý nghĩa: Bản tin "di chúc" (Last Will), Broker tự động phát khi camera mất điện/mất kết nối đột ngột.

video/ {camera_id}	AI Worker	Backend	Payload: Chuỗi string (Base64 định dạng JPG). Ý nghĩa: Đẩy luồng ảnh proxy tĩnh liên tục để Backend phát qua WebSocket lên màn hình giám sát trực tuyến của người dùng.
alerts/ ai_detected	AI Worker	Backend	Payload: {"camera_id": int, "temp_video": string, "timestamp": string}. Ý nghĩa: Phát tín hiệu khi YOLO phát hiện lửa vượt ngưỡng tin cậy. Server nhận bản tin này sẽ đối chiếu ngay với cảm biến vật lý để phát cảnh báo.
commands/ capture_alert _media/ {camera_id}	Backend	AI Worker	Payload: {"alert_id": int, "temp_video": string, "alert_time": string}. Ý nghĩa: Lệnh từ server yêu cầu AI Worker lưu vết bằng chứng. AI Worker nhận lệnh sẽ lập tức di chuyển ảnh tại $T = 0$ từ thư mục temp sang thư mục lưu trữ chính thức.
alerts/ image_ready	AI Worker	Backend	Payload: {"alert_id": int, "camera_id": int, "image_filename": string}. Ý nghĩa: AI Worker báo cáo đã lưu và đặt tên file ảnh chụp đám cháy thành công, server tiếp nhận để cập nhật trường dữ liệu ảnh tương ứng trong CSDL.

alerts/ video_ready	AI Worker	Backend	Payload: {"alert_id": int, "camera_id": int, "video_filename": string}. Ý nghĩa: Gửi lên báo cho server biết tệp video bằng chứng đã hoàn thành để lưu tên file vào CSDL.
------------------------	--------------	---------	--

Bảng 4.3: Danh mục đặc tả các Topic MQTT hệ thống

4.3 Thiết kế thiết bị vật lý (Camera)

Hệ thống sử dụng thiết bị ESP32 kết nối với camera và các thiết bị cảm biến, vận hành theo cơ chế đa nhiệm phi khóa (Non-blocking multitasking) sử dụng hàm millis() để đảm bảo tính thời gian thực cao nhất.

Trong môi trường thử nghiệm này, hệ thống sẽ bao gồm 2 bộ thiết bị camera:

- **Bộ số 1:** vi xử lý ESP32-CAM, các cảm biến gồm: cảm biến phát hiện ngọn lửa KY-026, cảm biến phát hiện khói MP-2.
- **Bộ số 2:** vi xử lý ESP32 (không CAM), các cảm biến gồm: cảm biến nhiệt độ - độ ẩm môi trường DHT11, cảm biến khí gas MQ2. Để giả lập camera, bộ số 2 sẽ sử dụng tạm thời camera của laptop.

Vì các bộ ADC trên ESP32-CAM bị giới hạn (do phải nhường cho camera và Wi-Fi hoạt động), nên các cảm biến trên bộ số 1 sẽ chỉ hoạt động ở chế độ Digital.

4.3.1 Sơ đồ kết nối chân phần cứng

Linh kiện ngoại vi	Chân ESP32	Chân ESP32-CAM	Chế độ chân (Pin Mode)
Cảm biến Lửa (KY026 - Digital)	-	GPIO 2	INPUT
Cảm biến Khói (MQ2 - Digital)	-	GPIO 13	INPUT (Đảo mức logic do tích cực thấp)

Cảm biến Nhiệt độ/Độ ẩm DHT11	GPIO 4	-	INPUT (Giao tiếp 1-wire)
Cảm biến Khí gas (MQ2 - Analog)	GPIO 34	-	INPUT (Đọc ADC 11dB)
Còi báo động & Đèn đỏ cảnh báo	GPIO 18	GPIO 14	OUTPUT (Tích cực cao)
Nút nhấn khôi phục (Reset)	GPIO 15	GPIO 12	INPUT_PULLUP (Tích cực thấp)
Đèn trạng thái (Status LED)	GPIO 2	GPIO 15	OUTPUT (Chớp phi khóa)

Bảng 4.4: Bảng sơ đồ kết nối chân phần cứng

4.3.2 Quy trình hoạt động

Chu trình xử lý cốt lõi của thiết bị bao gồm 4 quá trình được miêu tả như sau:

Đầu tiên, khi khởi động, vi điều khiển gọi thư viện WiFiManager để kiểm tra thông tin mạng cũ. Nếu trống, thiết bị tự phát một điểm truy cập WiFi (Access Point) để người dùng đăng nhập và cấu hình tên mạng cùng địa chỉ IP của server. Thông tin này sau đó được lưu trực tiếp vào Flash thông qua thư viện Preferences để tự động nạp lại cho các lần khởi động tiếp theo. Trong trường hợp muốn xóa sạch cấu hình để thiết lập lại từ đầu, người dùng chỉ cần nhấn giữ nút kết nối với chân RESET_PIN trong 5 giây.

Tiếp theo, định kỳ mỗi 5 giây, tiến trình checkSensors() sẽ tự động kích hoạt. Mạch ESP32 thực hiện đọc giá trị ADC từ cảm biến Gas cùng cảm biến nhiệt độ DHT11. Trong khi đó ESP32-CAM đọc trạng thái logic từ cảm biến khói và lửa. Dữ liệu sau đó được đóng gói thành chuỗi JSON và gửi lên topic sensors/macAddress. Ngoài ra, cứ mỗi 60 giây, thiết bị sẽ tự động gửi bản tin cấu hình phần cứng lên topic discovery/cameras để server nhận diện và cập nhật cấu hình.

Sau đó, thiết bị liên tục đăng ký lắng nghe trên topic lệnh commands/macAddress. Khi nhận được tín hiệu bật còi, hàm callback sẽ lập tức kích xuất điện áp mức cao ra chân BUZZER_PIN để báo động trực tiếp tại hiện trường. Song song với đó,

tiến trình ngầm `updateStatusLED()` liên tục quét môi trường mạng để nhảy đèn trạng thái: nhảy siêu tốc (100ms) khi mất WiFi, nhảy vừa (500ms) khi mất kết nối MQTT Broker, và nhảy chậm (2000ms) khi hệ thống vận hành ổn định.

Đối với ESP32-CAM, thiết bị sẽ đảm nhận thêm việc truyền phát hình ảnh. Thiết bị sẽ khởi chạy hai Endpoint HTTP riêng biệt phục vụ cho tầng xử lý ảnh. Đường dẫn `/mjpeg/1` thiết lập một vòng lặp liên tục đẩy ảnh dạng multipart về phía AI Worker, trong đó hàm điều phối `runBackgroundTasks()` được gọi đan xen để vi điều khiển vừa giữ được luồng video, vừa không bỏ sót lệnh MQTT hay chu kỳ đọc cảm biến. Đường dẫn `/jpg` đóng vai trò trả về một khung hình tĩnh duy nhất khi có yêu cầu, giúp server vẫn có thể lấy được ảnh đám cháy nếu tiến trình AI không hoạt động hoặc gặp sự cố.

4.4 Thiết kế chương trình xử lý AI

Chương trình dành cho tiến trình AI Worker được xây dựng bằng ngôn ngữ Python, kết hợp với thư viện OpenCV, mô hình học sâu YOLO và vận hành như một dịch vụ ngầm độc lập. Server khi hoạt động sẽ gọi một tiến trình AI Worker đảm nhiệm nhiệm vụ quản lý và phân tích hình ảnh cho mỗi camera khi nó đang ở trạng thái Online.

Đầu tiên, chương trình sử dụng thư viện OpenCV, lớp `VideoStreamReader` liên tục kéo luồng hình ảnh từ thiết bị với tần suất ổn định 10 FPS để tránh lãng phí CPU. Nhằm tối ưu phần cứng, mô hình học sâu YOLO chỉ thực hiện quét tìm khói và lửa sau mỗi chu kỳ 5 khung hình. Nếu độ tin cậy vượt ngưỡng cấu hình, cờ hiệu đám cháy `has_fire` sẽ được bật. Song song đó, mọi khung hình đều được nén độ phân giải về mức 320×240 và mã hóa thành chuỗi Base64 siêu nhẹ, rồi Publish liên tục lên topic `video/cameraId` để làm luồng video hiển thị trực tuyến trên giao diện.

Trong suốt quá trình vận hành, AI Worker liên tục đẩy các khung hình vào hàng đợi xoay vòng `frame_buffer` để lưu trữ tạm thời 60 giây dữ liệu mới nhất. Ngay khi cờ `has_fire` kích hoạt, chương trình chụp lại một ảnh tĩnh và gửi gói tin JSON lên topic `alerts/ai_detected` để máy chủ trung tâm xử lý và phát báo động khẩn cấp ngay lập tức. Cùng lúc, một luồng khác độc lập được khởi chạy, tự động "ngủ" đúng 30 giây để chờ luồng chính tiếp tục nạp các khung hình diễn biến tương lai vào bộ đệm. Khi thức dậy, luồng ngầm tiến hành trích xuất dữ liệu

để đóng gói thành tệp video .webm chứa trọn vẹn sự cố bao gồm 30 giây quá khứ và 30 giây tương lai.

Để tránh rác dữ liệu trên ổ cứng do AI nhận diện sai, toàn bộ tệp ảnh và video ban đầu đều được lưu vào phân vùng cách ly tạm thời tại thư mục `saved_images/temp` và `saved_videos/temp`. AI Worker đăng ký lắng nghe trên topic lệnh `commands/capture_alert_media/cameraId`. Khi nhận được tín hiệu xác nhận đám cháy thật từ Backend gửi xuống, hệ thống lập tức chuyển tệp ảnh sang thư mục lưu trữ chính thức và gửi tín hiệu hoàn tất vào topic `alerts/image_ready`. Đối với tệp video, nếu luồng ngấm chưa kết xuất xong, lệnh sẽ được đưa vào hàng đợi `pending_commands` và tự động xử lý chuyển vùng sang thư mục chính thức ngay khi video ghi nhận thành công, sau đó gửi tín hiệu lên topic `alerts/video_ready` để hoàn thành chu trình.

4.5 Thiết kế cơ sở dữ liệu

Từ sơ đồ lớp của hệ thống như trên, hệ thống cơ sở dữ liệu được xây dựng với các bảng gồm các thuộc tính được liệt kê ở dưới đây:

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, Auto Increment	Khóa chính, định danh duy nhất của người dùng.
username	VARCHAR	UNIQUE, NOT NULL	Tên đăng nhập hệ thống.
email	VARCHAR	UNIQUE, NOT NULL	Địa chỉ thư điện tử dùng để liên lạc và xác thực.
name	VARCHAR	NOT NULL	Họ và tên hiển thị của người dùng.
password_hash	VARCHAR	NOT NULL	Mật khẩu đã được băm (bcrypt) để bảo mật.
status	ENUM	NOT NULL, DEFAULT 'active'	Trạng thái tài khoản (active, inactive, deleted).

fcm_tokens	JSONB	DEFAULT '[]'	Mảng lưu các mã token FCM định danh thiết bị di động để nhận thông báo đẩy.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo tài khoản.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật thông tin tài khoản gần nhất.
deleted_at	TIMESTAMP	NULL	Thời điểm xóa mềm tài khoản.

Bảng 4.5: Đặc tả chi tiết bảng users

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, Auto Increment	Khóa chính, định danh duy nhất của phòng.
name	VARCHAR	NOT NULL	Tên khu vực/phòng (Ví dụ: Nhà kho, Phòng khách).
description	VARCHAR	NULL	Mô tả chi tiết về đặc điểm phòng.
owner_id	INT	FK, NOT NULL	Khóa ngoại tham chiếu đến id của bảng users (Chủ sở hữu phòng).
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm khởi tạo khu vực.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm chỉnh sửa thông tin khu vực gần nhất.

Bảng 4.6: Đặc tả chi tiết bảng rooms

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, Auto Increment	Khóa chính, định danh duy nhất của thiết bị trên server.
mac_address	VARCHAR	UNIQUE, NOT NULL	Địa chỉ MAC vật lý của thiết bị phần cứng, dùng để đồng bộ mạng.
ip_address	VARCHAR	NULL	Địa chỉ IP hiện tại của camera trong mạng cục bộ.
name	VARCHAR	NOT NULL	Tên gọi nhớ của thiết bị.
stream_url	VARCHAR	NULL	Đường dẫn URL để kéo luồng truyền phát video trực tiếp.
description	VARCHAR	NULL	Ghi chú thêm về thiết bị.
status	ENUM	NOT NULL, DEFAULT 'online'	Trạng thái kết nối hiện tại (not_binded, online, offline).
room_id	INT	FK, NULL	Khóa ngoại tham chiếu bảng rooms (ON DELETE SET NULL).
owner_id	INT	FK, NOT NULL	Khóa ngoại tham chiếu bảng users (ON DELETE CASCADE).
yolo_confidence	FLOAT	NULL	Ngưỡng tin cậy (Confidence) tối thiểu để AI xác nhận có khói/lửa.
max_temperature	FLOAT	NULL	Ngưỡng nhiệt độ tối đa cho phép báo động (Đơn vị: °C).
max_gas_percent	FLOAT	NULL	Ngưỡng nồng độ khí gas tối đa cho phép báo động (Đơn vị: %).

smoke_trigger	BOOLEAN	NULL	Cờ kích hoạt/vô hiệu hóa việc báo động từ cảm biến khói cứng.
flame_trigger	BOOLEAN	NULL	Cờ kích hoạt/vô hiệu hóa việc báo động từ cảm biến lửa cứng.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm thêm thiết bị vào hệ thống.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật cấu hình thiết bị gần nhất.

Bảng 4.7: Đặc tả chi tiết bảng cameras

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, Auto Increment	Khóa chính, định danh của bản ghi cảnh báo.
camera_id	INT	FK, NOT NULL	Khóa ngoại tham chiếu tới thiết bị phát hiện sự cố (bảng cameras) (ON DELETE CASCADE).
level	ENUM	NOT NULL, DEFAULT 'high'	Mức độ nghiêm trọng của báo động (low, medium, high).
image_filename	VARCHAR	NULL	Tên tệp tin hình ảnh chụp tại khoảnh khắc xảy ra đám cháy.
video_filename	VARCHAR	NULL	Tên tệp tin video ghi hình sự cố đám cháy.
is_resolved	BOOLEAN	NOT NULL, DEFAULT FALSE	Trạng thái xử lý (True: Đã xử lý an toàn; False: Chưa xử lý).

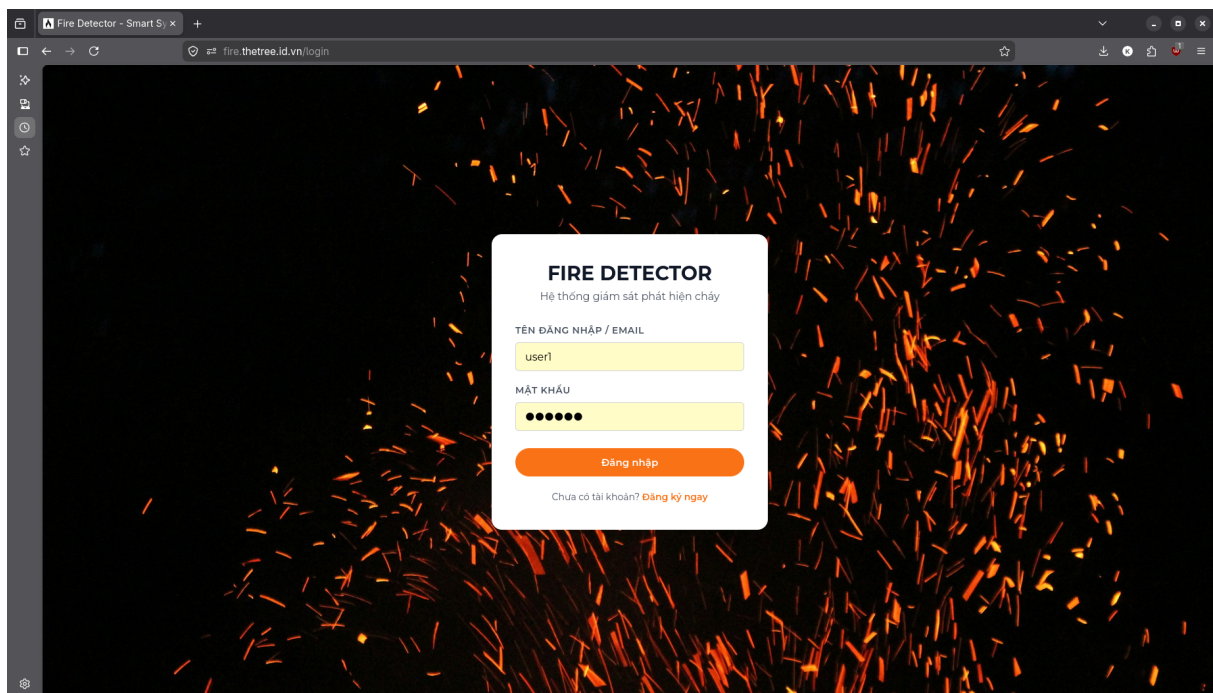
detected_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Mốc thời gian chính xác hệ thống xác định có cháy.
-------------	-----------	-------------------------------	---

Bảng 4.8: Đặc tả chi tiết bảng *fire_alerts*

4.6 Thiết kế giao diện

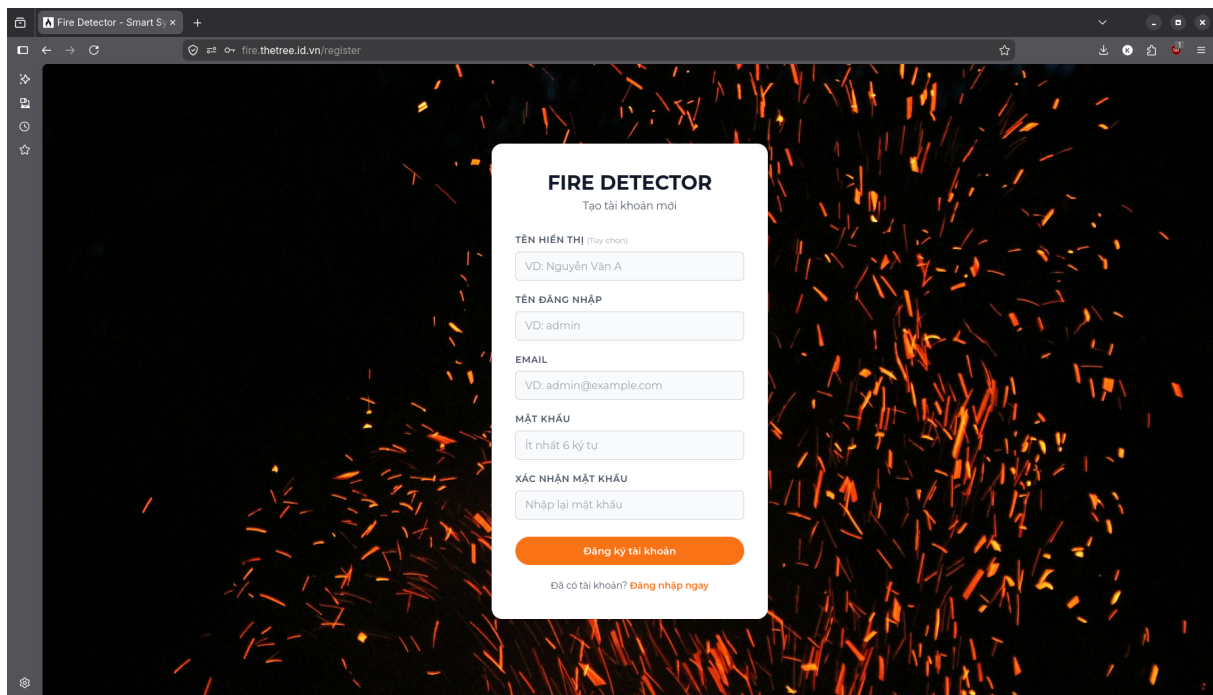
4.6.1 Giao diện đăng nhập & đăng ký tài khoản

Giao diện đăng nhập được thiết kế với hai trường Tên đăng nhập/Email và Mật khẩu. Khi người dùng đăng nhập thành công, hệ thống sẽ điều hướng vào trang dashboard chính.



Hình 4.2: Giao diện đăng nhập

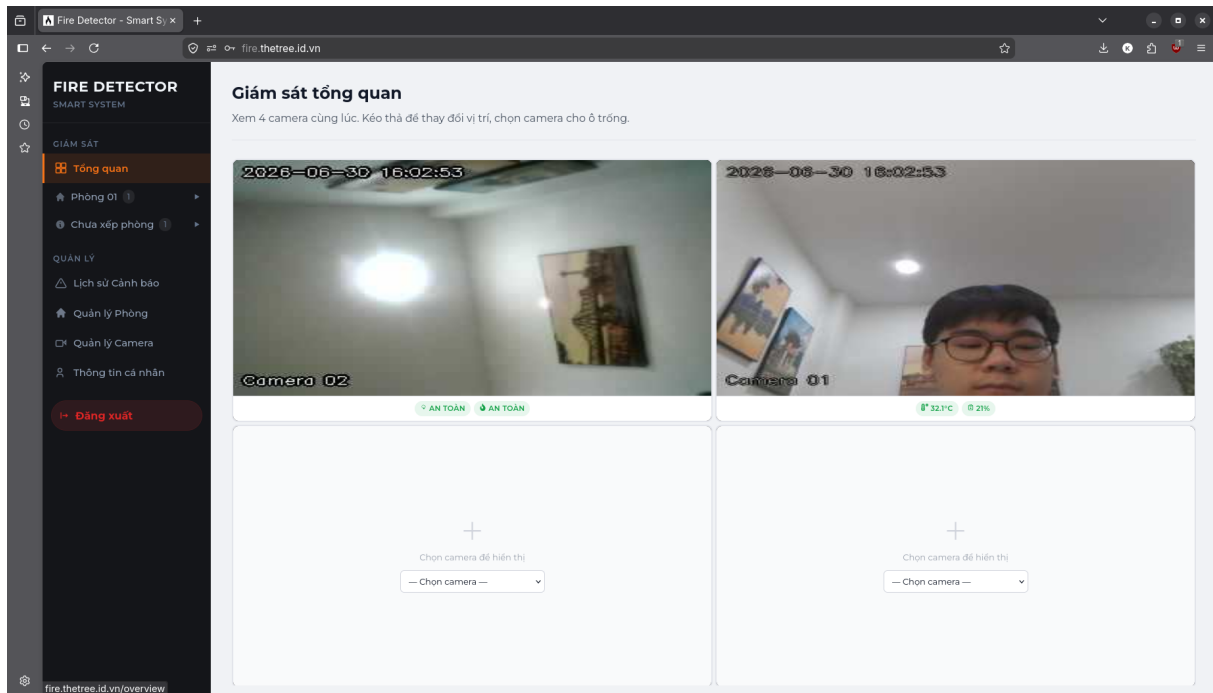
Nếu người dùng chưa có tài khoản, có thể chuyển sang giao diện đăng ký tài khoản. Tại đây, người dùng khai báo các thông tin như theo yêu cầu



Hình 4.3: *Giao diện đăng ký tài khoản*

4.6.2 Giao diện giám sát tổng quan

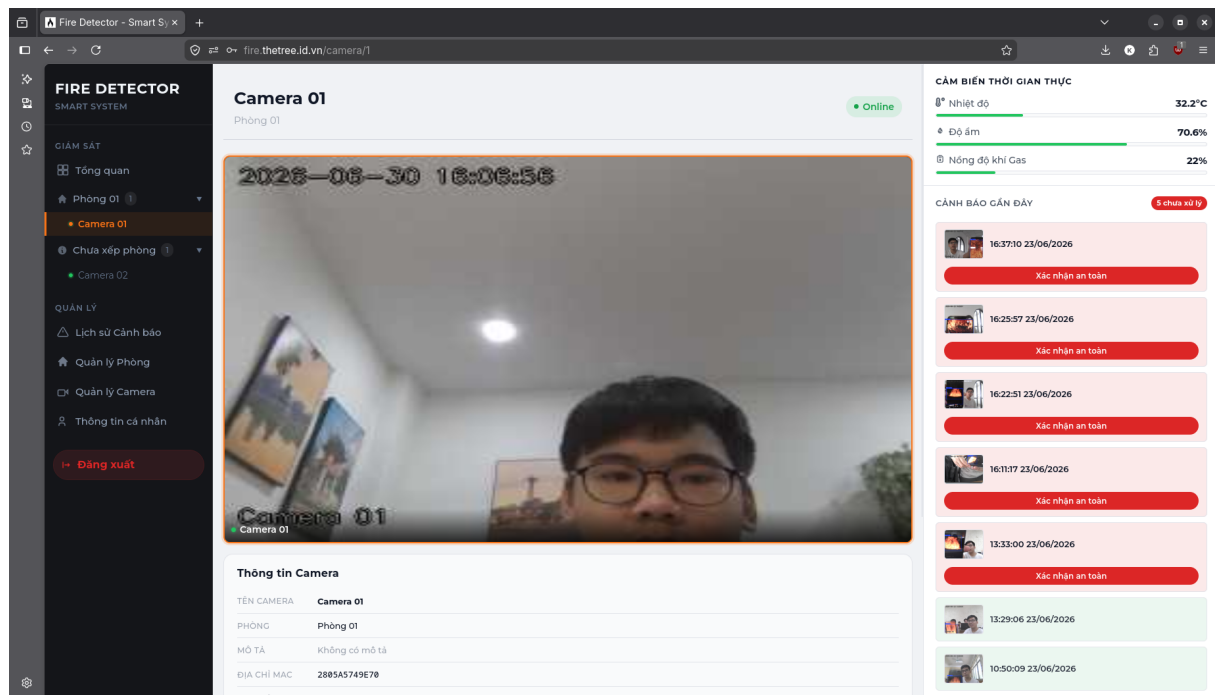
Tại giao diện theo dõi tổng quan, người dùng có thể theo dõi 4 camera cùng lúc, đi kèm với đó là các thông số cảm biến. Người dùng có thể kéo thả, lựa chọn camera cho từng ô nhỏ. Khi có cháy xảy ra, ô camera tương ứng cũng sẽ hiển thị cảnh báo.



Hình 4.4: *Giao diện giám sát tổng quan*

4.6.3 Giao diện giám sát từng camera

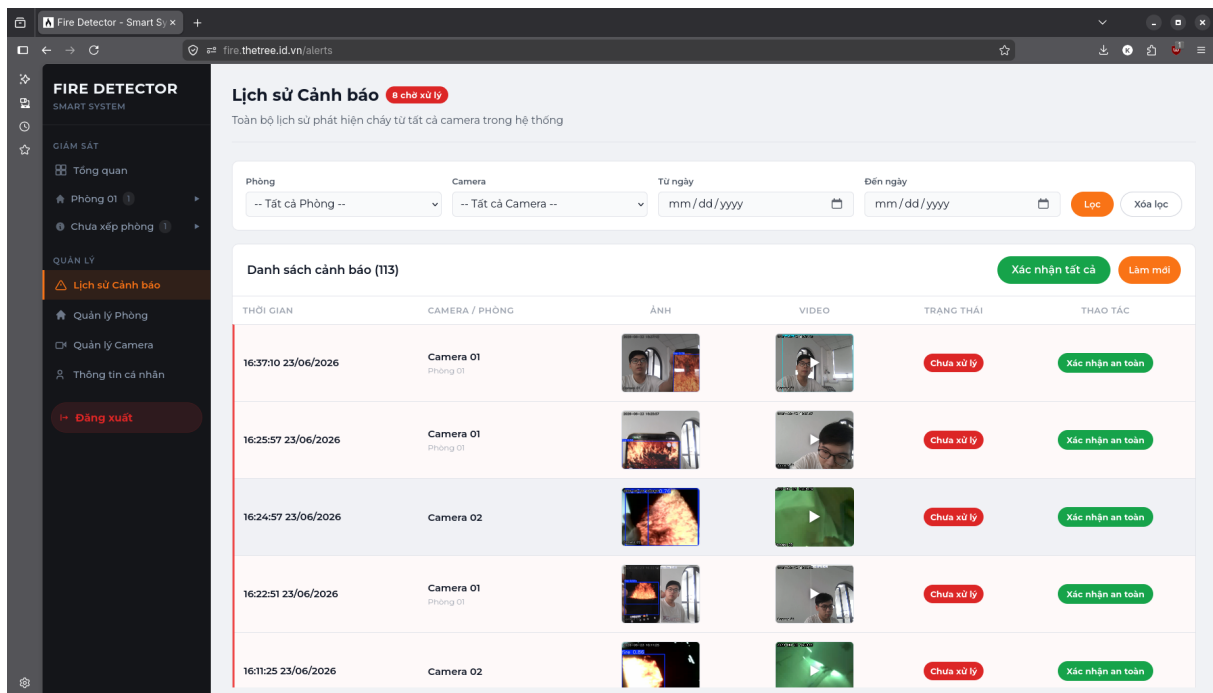
Danh sách camera được phân chia theo các phòng, bao gồm cả các camera chưa được xếp phòng. Tại đây, người dùng có thể theo dõi hình ảnh camera, thông số cảm biến theo thời gian thực, cũng như các thông tin, cấu hình và trạng thái camera. Bên cạnh đó là danh sách các cảnh báo trong lịch sử, đi kèm với đó là nút xác nhận an toàn đối với các cảnh báo chưa xác nhận.



Hình 4.5: Giao diện giám sát từng camera

4.6.4 Giao diện lịch sử cảnh báo

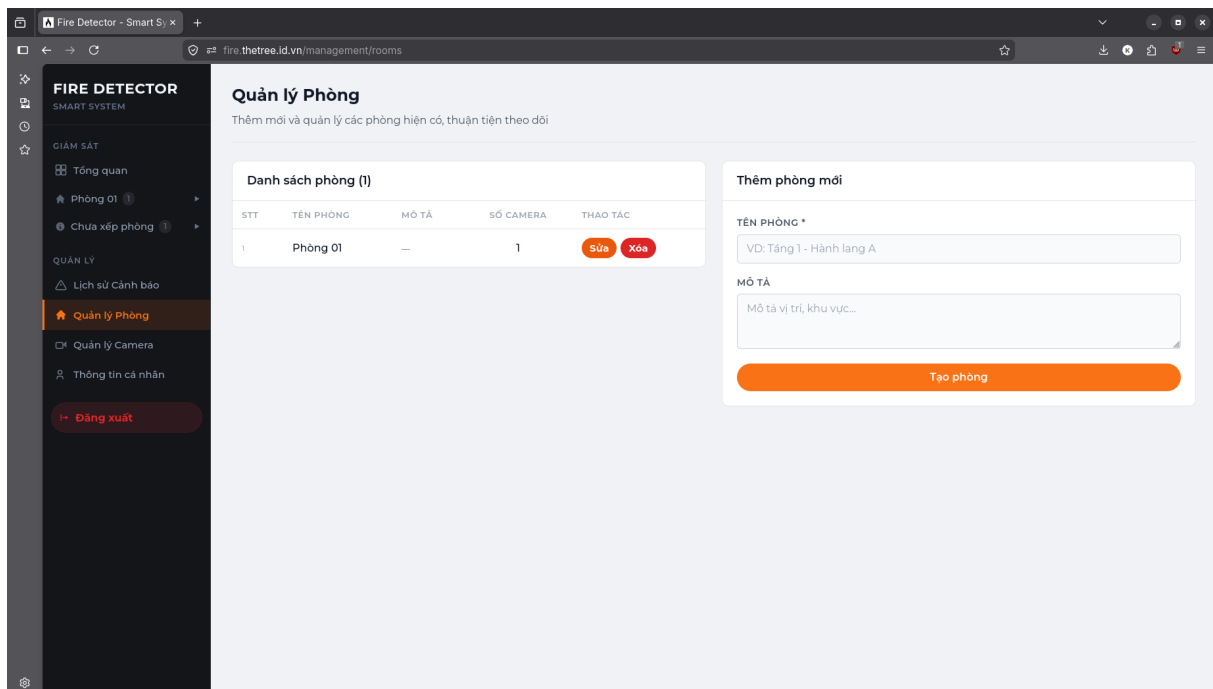
Người dùng có thể theo dõi toàn bộ lịch sử các cảnh báo của hệ thống ở giao diện Lịch sử cảnh báo, với các thông tin cảnh báo như: thời gian, camera phát hiện, hình ảnh/video đám cháy, trạng thái. Cùng với đó là nút xác nhận an toàn nếu cảnh báo chưa xác nhận. Người dùng cũng có thể tìm kiếm, lọc cảnh báo, và xác nhận an toàn cho tất cả cảnh báo.



Hình 4.6: Giao diện lịch sử cảnh báo

4.6.5 Giao diện quản lý phòng

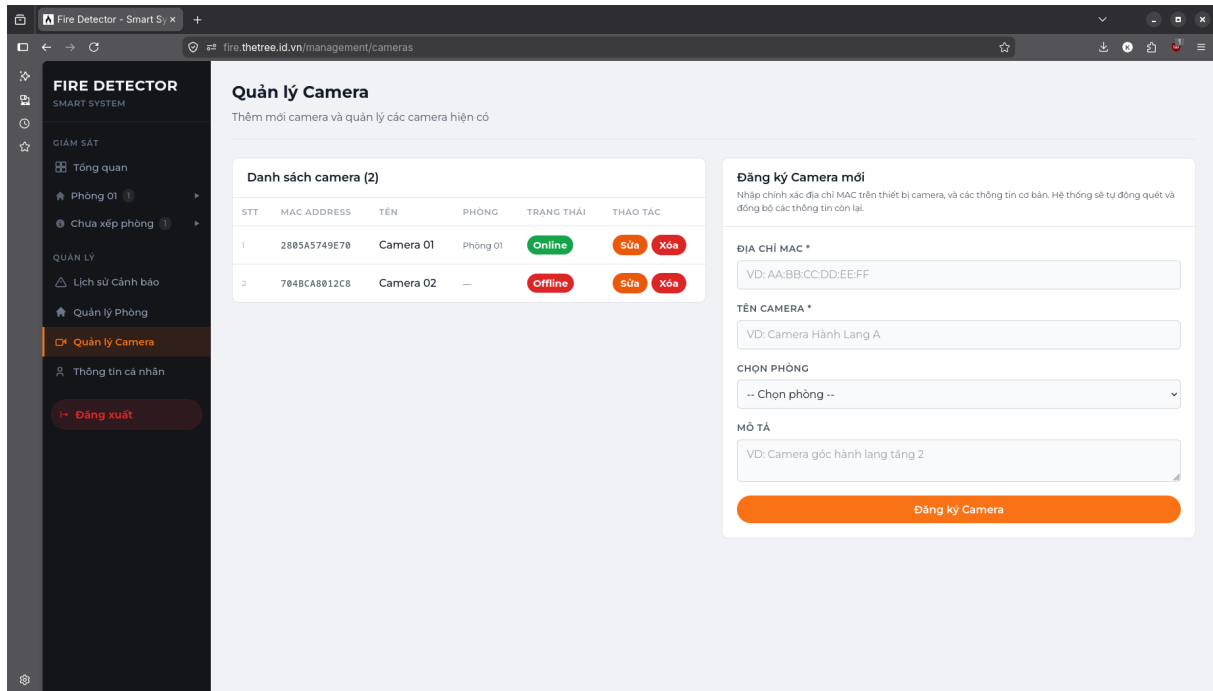
Tại giao diện quản lý phòng, người dùng có thể xem danh sách tất cả các phòng hiện có của mình, tạo mới phòng, cũng như chỉnh sửa các phòng hiện có



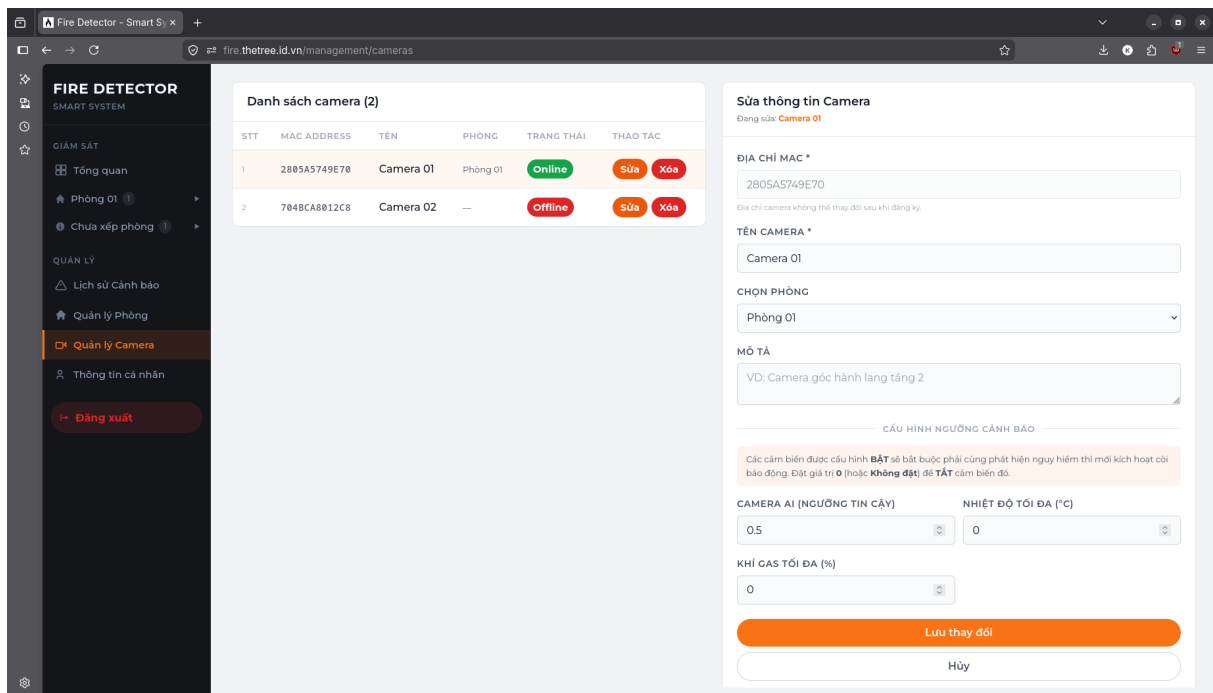
Hình 4.7: Giao diện quản lý phòng

4.6.6 Giao diện quản lý camera

Tương tự, tại giao diện quản lý camera, người dùng có thể xem danh sách tất cả các camera hiện có của mình. Để thêm camera mới, người dùng chỉ cần nhập địa chỉ MAC của camera (địa chỉ này là duy nhất với mỗi camera), và các thông tin cơ bản. Các thông số còn lại hệ thống sẽ tự động bộ. Để chỉnh sửa các thông số ngưỡng cảm biến, người dùng có thể truy cập qua giao diện sửa camera.



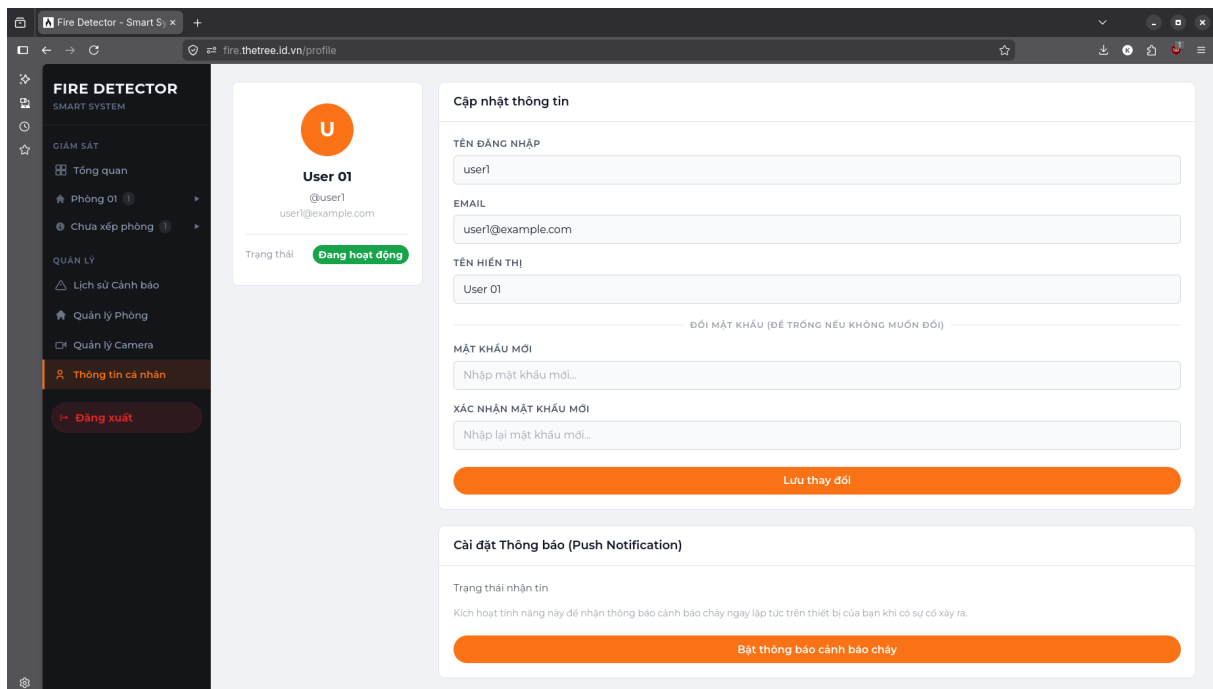
Hình 4.8: Giao diện quản lý camera



Hình 4.9: Giao diện chỉnh sửa camera

4.6.7 Giao diện thông tin cá nhân

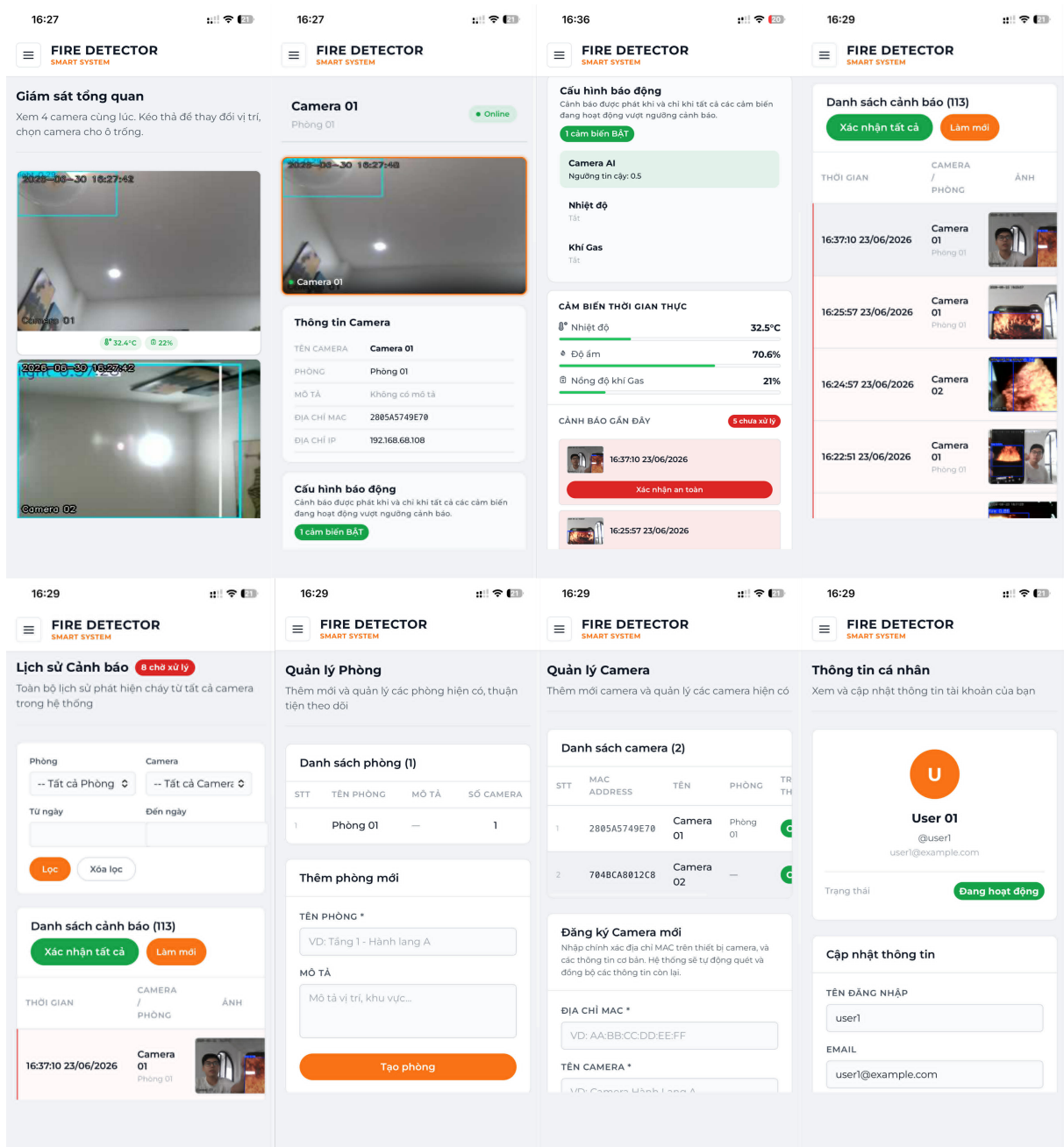
Tại đây, người dùng có thể xem và điều chỉnh thông tin cá nhân, đổi mật khẩu, và bật tính năng cảnh báo đầy cho thiết bị.



Hình 4.10: Giao diện thông tin cá nhân

4.6.8 Giao diện dành cho điện thoại

Ngoài giao diện Web dành cho máy tính bàn ở trên, hệ thống cũng cung cấp giao diện thân thiện dành cho người dùng di động, với các thành phần giao diện tương tự.

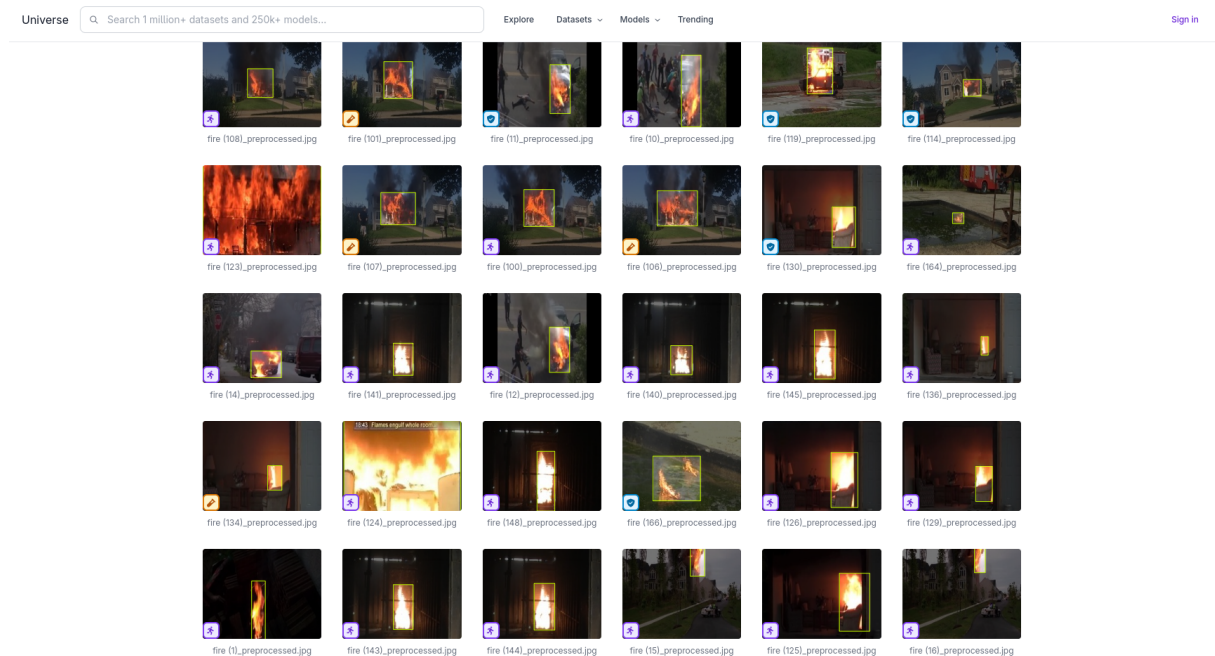


Hình 4.11: Giao diện dành cho điện thoại

CHƯƠNG 5: KẾT QUẢ THỰC HIỆN

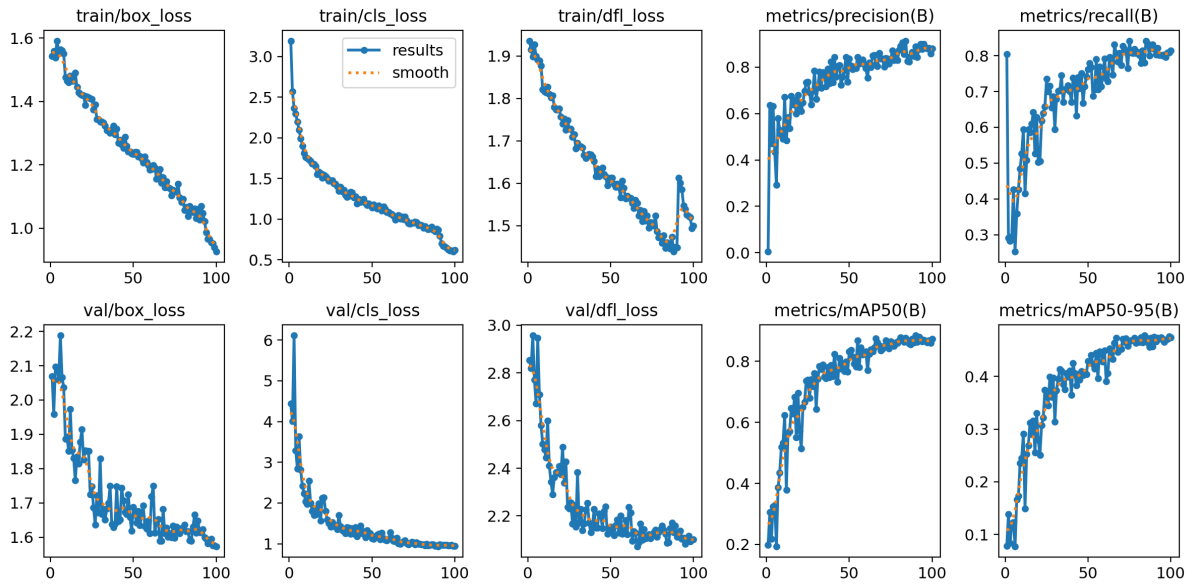
5.1 Kết quả huấn luyện mô hình YOLO

Nguồn dữ liệu huấn luyện được thu thập từ Roboflow ([Link](#)) với tổng cộng 1542 bức ảnh đa dạng về môi trường, với 1079 bức ảnh dành cho tập huấn luyện, 309 bức cho tập đánh giá và 154 bức cho tập kiểm thử. Dữ liệu cũng được phân loại và gán nhãn cho 4 lớp đối tượng là fire (lửa), smoke (khói), light (ánh sáng) và no-fire (không có cháy).



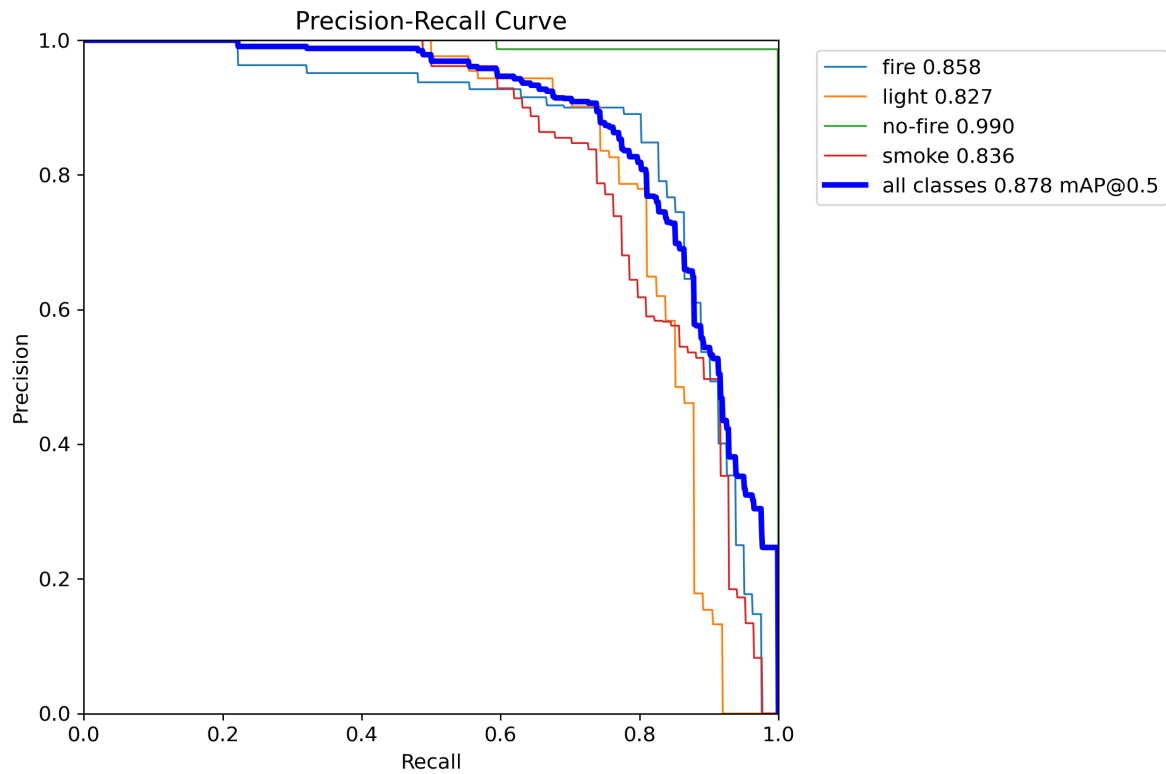
Hình 5.1: Dữ liệu huấn luyện trên Roboflow

Quá trình huấn luyện mô hình diễn ra ổn định, được thể hiện rõ qua biểu đồ các hàm mất mát. Các chỉ số mất mát trên tập huấn luyện bao gồm train/box_loss (sai số hộp bao), train/cls_loss (sai số phân loại) và train/df_l_loss đều thể hiện xu hướng giảm đều đặn và tiệm cận về mức tối thiểu qua mỗi chu kỳ huấn luyện. Đặc biệt, các đường biểu diễn hàm mất mát trên tập đánh giá (val/box_loss và val/cls_loss) luôn bám sát với đường biểu diễn của tập huấn luyện. Kết quả này chứng minh mô hình đã hội tụ, có khả năng suy luận tốt trên dữ liệu mới mà không gặp phải hiện tượng overfitting.



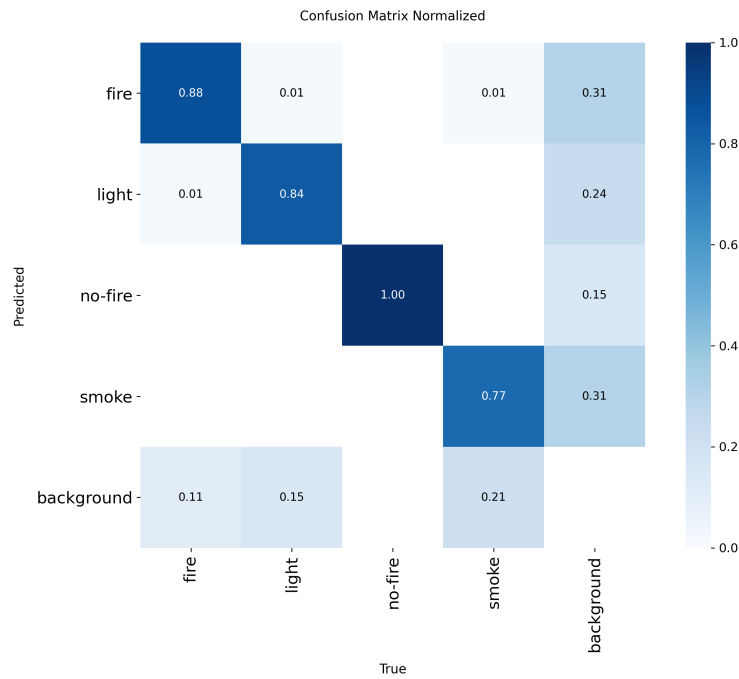
Hình 5.2: Kết quả huấn luyện mô hình YOLO

Kết quả trích xuất từ lịch sử huấn luyện, tại chu kì thứ 91 mô hình đạt độ chính xác trung bình mAP50 ở mức 86.4%. Chỉ số Precision và Recall lần lượt đạt 84.3% và 80.2%, tức là khoảng 84% các dự báo là chuẩn xác và phát hiện được khoảng 80% các đối tượng thật. Như vậy, mô hình có khả năng phát hiện chuẩn xác và không bỏ sót các dấu hiệu hỏa hoạn; giúp hạn chế tối đa tình trạng báo động giả gây phiền toái cho người dùng.



Hình 5.3: Đường cong PR Curve

Phân tích ma trận nhầm lẫn, có thể thấy mô hình cho thấy khả năng nhận diện tốt đối với lớp fire với tỷ lệ dự đoán đúng lên tới 88%. Điều đó chứng tỏ mô hình đã phân tách thành công các đặc trưng cốt lõi của hỏa hoạn so với các yếu tố gây nhiễu từ môi trường xung quanh.



Hình 5.4: Ma trận nhầm lẫn của mô hình

Như vậy, mô hình sau khi huấn luyện đã đạt được kết quả tốt, đủ điều kiện để đưa vào vận hành trong hệ thống.

5.2 Thử nghiệm hệ thống

5.2.1 Thử nghiệm với đám cháy giả lập

Tiến hành giả lập với đám cháy ”ảo” bằng hình ảnh, tạm thời tắt các cảm biến vật lý và chỉ bật báo động từ AI, hệ thống ngay lập tức phát báo động.



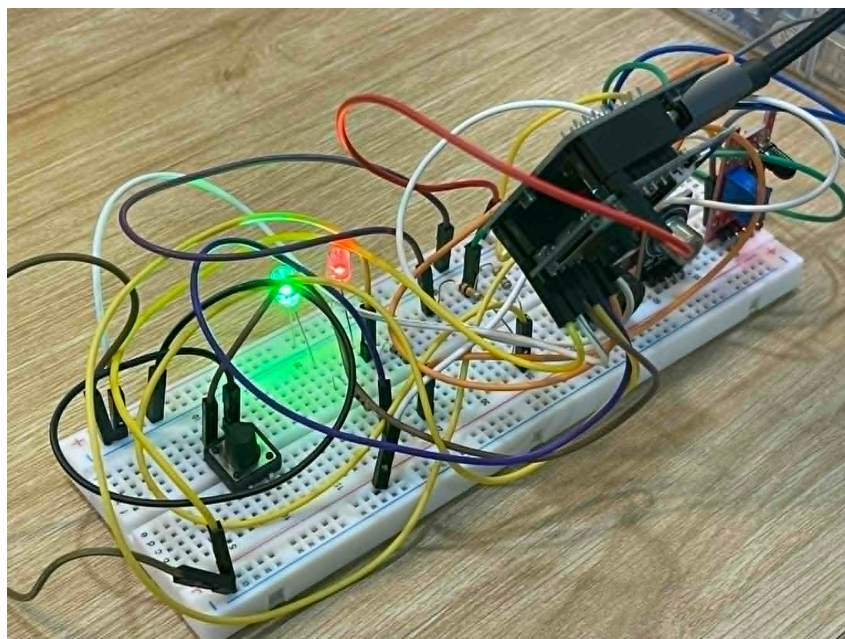
Hình 5.5: Hệ thống phát cảnh báo

Trên điện thoại, cảnh báo đầy cũng được phát.



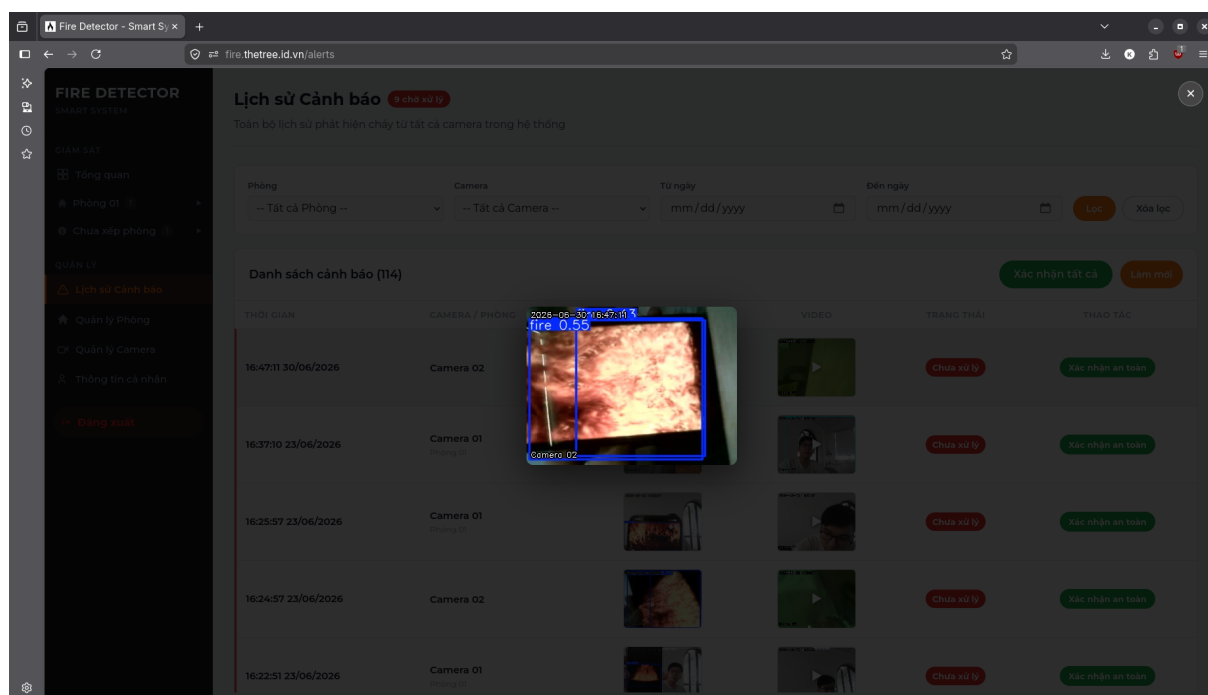
Hình 5.6: Cảnh báo đầy trên điện thoại

Đồng thời, tín hiệu còi báo và đèn báo (màu đỏ) trên camera vật lý cũng được phát. Như vậy, hệ thống đã hoạt động thành công với kịch bản đám cháy giả lập.



Hình 5.7: Camera vật lý cũng phát tín hiệu cảnh báo

Hình ảnh đám cháy được chụp lại và lưu lại trên hệ thống:

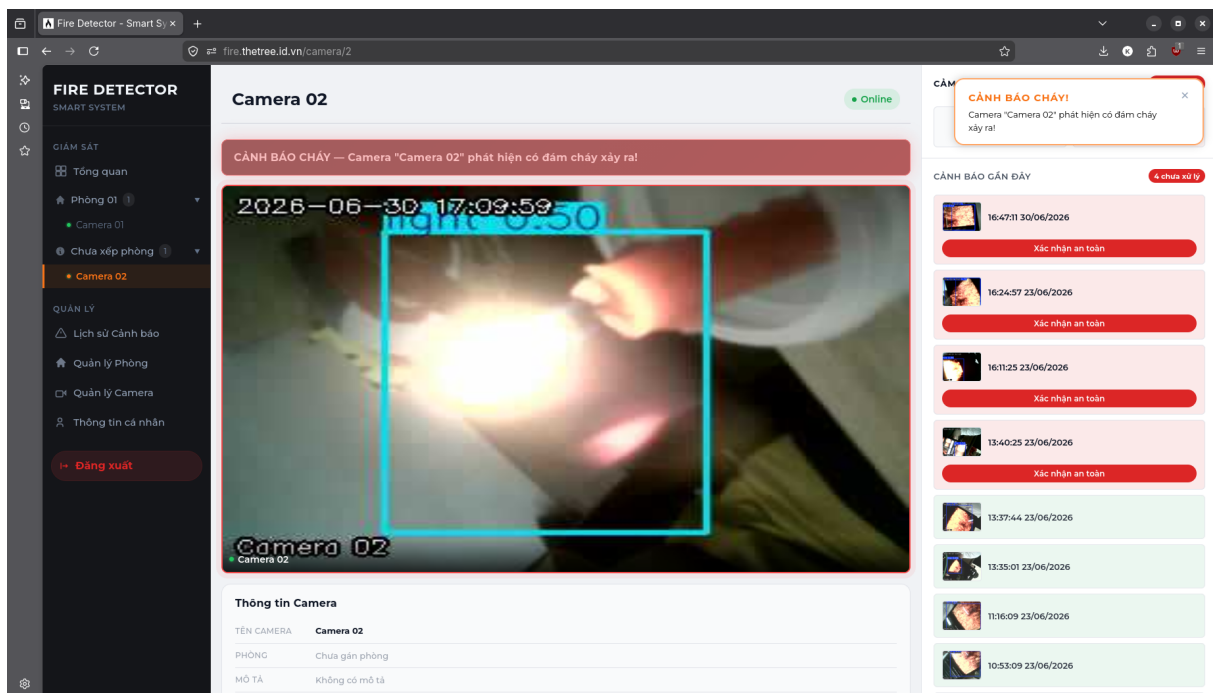


Hình 5.8: Hình ảnh đám cháy ghi nhận trên hệ thống

5.2.2 Thử nghiệm với đám cháy thật quy mô nhỏ

Tiến hành bật cảm biến lửa và nhận diện AI, nhưng tắt cảm biến khói; và thử nghiệm với đám cháy nhỏ, cụ thể là ngọn lửa từ cây nến nhỏ. Hệ thống cũng phát hiện cháy và phát tín hiệu báo động.

Tuy nhiên, có thể thấy mô hình YOLO đôi lúc vẫn nhận diện nhầm ngọn lửa thành nguồn ánh sáng. Nguyên nhân là do nguồn dữ liệu huấn luyện cho mô hình hầu hết là các đám cháy lớn, nên mô hình YOLO vẫn sẽ có đôi chút "lúng túng" khi xử lý với các ngọn lửa nhỏ. Đồng thời, chất lượng hình ảnh thấp do giới hạn phần cứng của camera cũng phần nào đó dẫn đến sự sai lệch của mô hình.



Hình 5.9: Giao diện người dùng phát cảnh báo



Hình 5.10: *Camera phát cảnh báo đèn đỏ*

CHƯƠNG 6: ĐÁNH GIÁ VÀ KẾT LUẬN

6.1 Kết quả đã đạt được

Trải qua quá trình nghiên cứu, phân tích thiết kế và triển khai thực tế, đồ án đã hoàn thành về cơ bản các mục tiêu đề ra ban đầu, xây dựng thành công một hệ thống cảnh báo cháy nổ thông minh tích hợp giữa IoT và AI.

Về mặt hạ tầng phần cứng, hệ thống đã tích hợp thành công bo mạch vi điều khiển kết hợp với các cảm biến vật lý như nhiệt độ, khói, lửa, khí gas nhằm phục vụ công tác thu thập dữ liệu môi trường tại hiện trường và thiết lập luồng truyền phát video trực tuyến ổn định từ ống kính thiết bị. Đồng thời là các tín hiệu cảnh báo, đèn LED được phát tại chỗ khi có tín hiệu phát hiện đám cháy, nhằm giúp người dùng tại nơi xảy ra đám cháy có thể phát hiện kịp thời và dập tắt đám cháy tại chỗ.

Về mặt lõi xử lý thông minh, đồ án đã áp dụng và huấn luyện thành công mô hình nhận diện thị giác máy tính YOLO chuyên biệt cho nhiệm vụ phát hiện khói và lửa. Tiến trình AI này được tối ưu hóa bằng thông nhờ cơ chế nén ảnh Proxy Base64 nhằm hạn chế độ trễ hiển thị trên giao diện.

Về phần server và ứng dụng, hệ thống đã áp dụng thành công framework NestJS theo cấu trúc các module gọn gàng, dùng TypeORM để kết nối và lưu trữ dữ liệu vào cơ sở dữ liệu PostgreSQL an toàn. Đồng thời kết hợp linh hoạt các giao thức: RESTful API để làm các chức năng quản lý, MQTT để giao tiếp giữa các thiết bị phần cứng, và WebSocket để gửi trực tiếp thông số cảm biến lên màn hình điện thoại/máy tính của người dùng theo thời gian thực. Bên cạnh đó là tích hợp dịch vụ Firebase Cloud Messaging, giúp điện thoại nhận được thông báo ngay lập tức.

6.2 Đánh giá hệ thống

Hệ thống đã đạt được nhiều ưu điểm lớn. Đầu tiên, tốc độ phản ứng nhanh, nhờ việc sử dụng giao thức MQTT và WebSocket, kết hợp với việc server phát lệnh báo động ngay khi phát hiện cháy. Thứ hai, đó là hạn chế báo động giả. Logic

xử lý tại máy chủ yêu cầu sự đồng thuận của cả cảm biến vật lý phần cứng lẫn kết quả phân tích hình ảnh từ mô hình AI, giúp hệ thống đưa ra cảnh báo chính xác hơn. Thứ ba, đó là xây dựng phương án lưu bằng chứng tốt. Việc tự động tách và lưu một bức ảnh tĩnh ngay tại khoảnh khắc xảy ra đám cháy giúp người dùng có ngay hình ảnh trực quan trên điện thoại để kịp thời đưa ra quyết định xử lý.

Tuy nhiên, hệ thống vẫn còn nhiều nhược điểm. Đầu tiên, kiến trúc triển khai còn phụ thuộc. Việc cài đặt chung nhiều thành phần trên cùng một máy chủ vật lý tạo ra rủi ro lớn trong trường hợp máy chủ gặp sự cố sập nguồn hoặc nghẽn mạng. Thứ hai, đó là mô hình AI vẫn còn thiếu chính xác khi nhận diện các đám cháy nhỏ, cũng như các cảm biến vật lý vẫn hiện chỉ có thể nhận diện được các đám cháy ở phạm vi gần và quy mô lớn. Cuối cùng, đó là giới hạn khả năng mở rộng. Tiến trình phân tích YOLO là tác vụ tiêu tốn nhiều tài nguyên hệ thống. Khi số lượng camera tăng lên, việc dùng chung một máy chủ sẽ dễ gây ra tình trạng quá tải cho máy chủ.

6.3 Hướng phát triển và mở rộng của đề tài

Từ những hạn chế được rút ra trong quá trình xây dựng và thử nghiệm hệ thống, em đề xuất một số hướng phát triển tiếp theo cho đề tài, nhằm giúp cho hệ thống hoàn thiện hơn.

Thứ nhất, tách riêng MQTT Broker lên cụm cloud độc lập và chuyển tầng xử lý AI sang một cụm server GPU chuyên dụng để chia tải, giúp hệ thống có thể mở rộng và quản lý nhiều camera cùng lúc.

Thứ hai, tối ưu hóa mô hình YOLO sang các định dạng nhẹ hơn để chạy trực tiếp trên các bo mạch phần cứng có chip tăng tốc AI. Việc nhận diện cháy ngay tại biên giúp hệ thống tiết kiệm băng thông mạng và có thể phát lệnh hú còi khẩn cấp hoàn toàn ngoại tuyến kể cả khi mất kết nối Internet.

Thứ ba, nâng cấp thuật toán nhận diện và các cảm biến vật lý. Cụ thể là cải thiện tập dữ liệu huấn luyện đa dạng và chính xác hơn, cũng như nghiên cứu và tích hợp các thuật toán phân tích hình ảnh theo chuỗi thời gian để phân biệt rõ ngọn lửa hỏa hoạn thực sự với các nguồn sáng gây nhiễu từ môi trường (như đèn LED, màn hình tivi, ánh đèn xe), nâng độ tin cậy của hệ thống lên mức tối đa. Đồng thời, thay thế các cảm biến hiện có sang các cảm biến hiện đại hơn, với phạm vi bao phủ rộng hơn, phù hợp với các nơi có không gian diện tích lớn.